NOVA IMS INFORMATION MANAGEMENT SCHOOL

Attention & Transformers

Text Mining 2025-2026

Bruno Jardim
bjardim@novaims.unl.pt

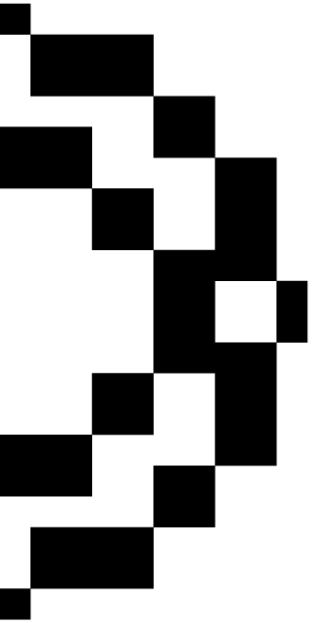
Lecture Plan

01. Review of Sequence-to-Sequence Models

02. Attention Mechanism

03. Transformer Architecture

04. Transformer Encoders

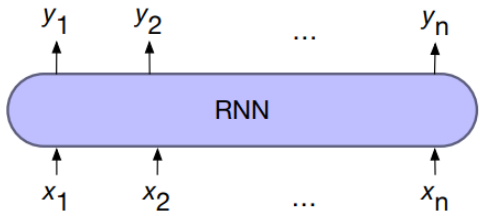


01.
Review of Seq2Seq Models

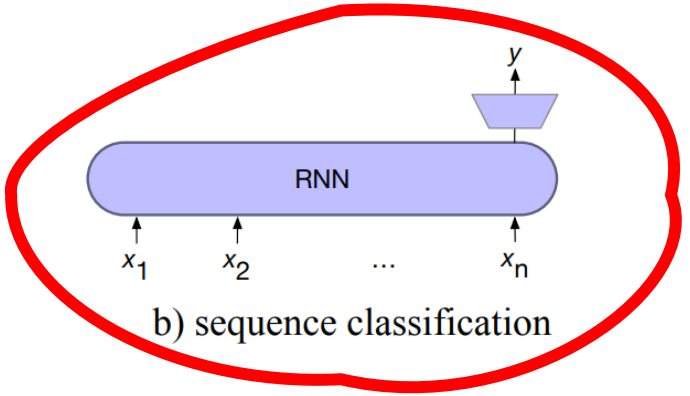
To **process sequential input**, we can use some well know sequential models, like **Recurrent Neural Networks (RNN)** and **Long Short Term Memory Neural Networks (LSTM)**

Common Tasks

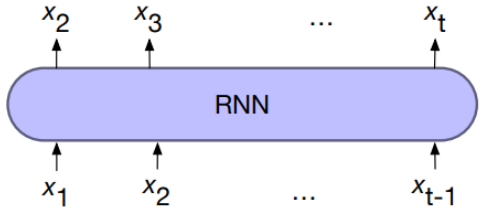
- Next word prediction
- Sequence Labeling
- **Sequence Classification**
- Machine Translation



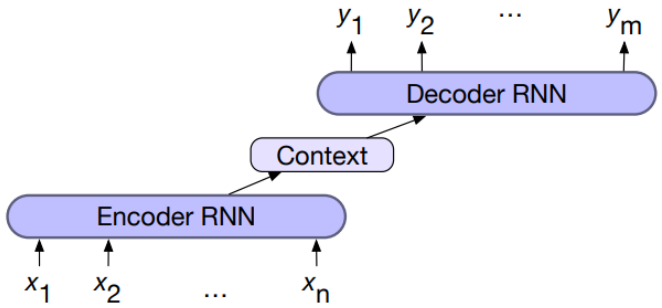
a) sequence labeling



b) sequence classification



c) language modeling

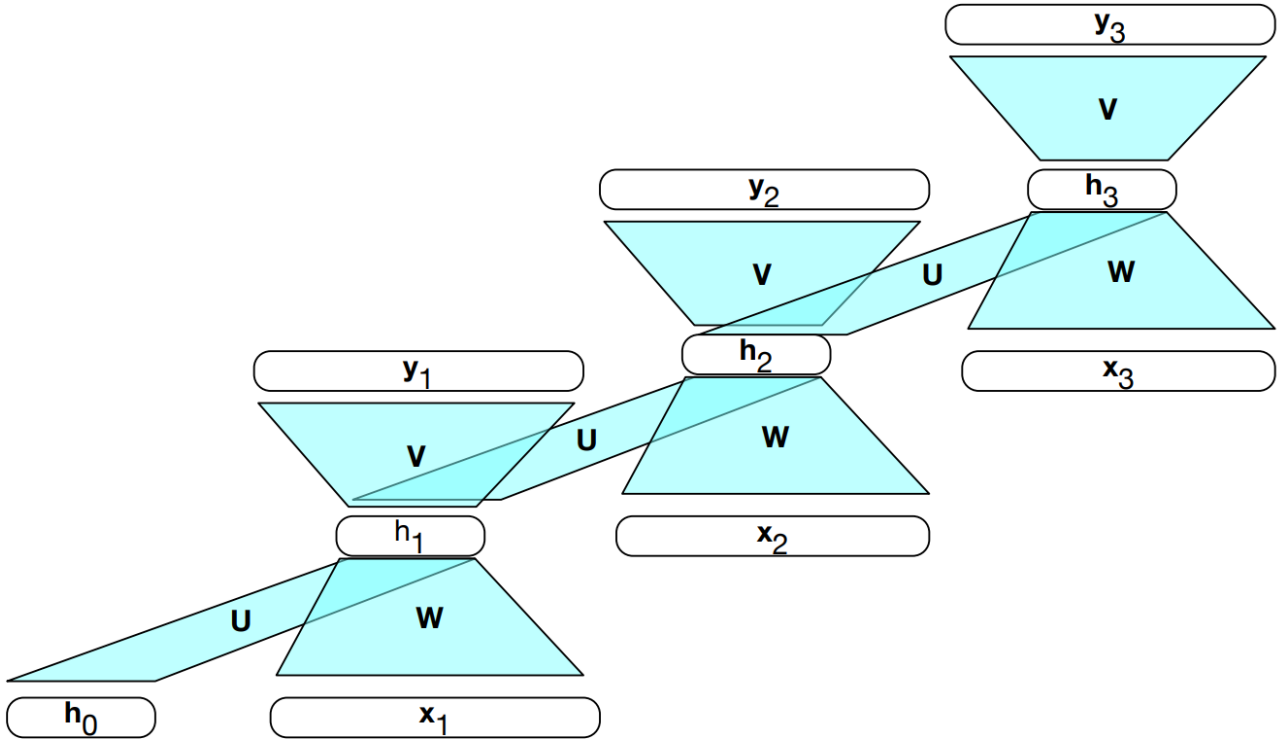


d) encoder-decoder

A **Recurrent Neural Network (RNN)** is a network that contains a cycle within its network connections, meaning that **the value of some unit is directly, or indirectly, dependent on its own earlier outputs** as an input.

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$$
$$\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t)$$

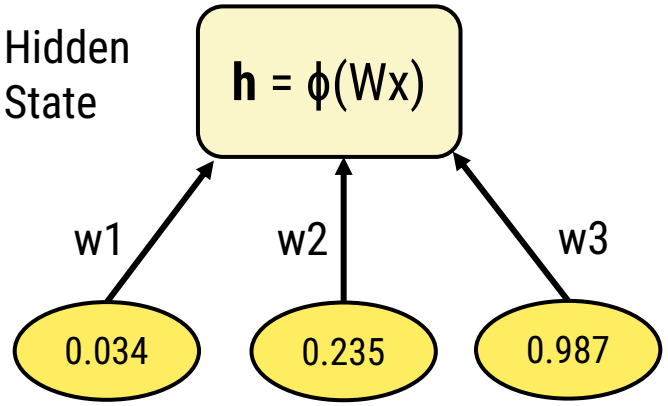
The matrices U, V and W are shared across time, while new values for h and y are calculated with each time step



Example:

Review 1 – "**Great** movie"

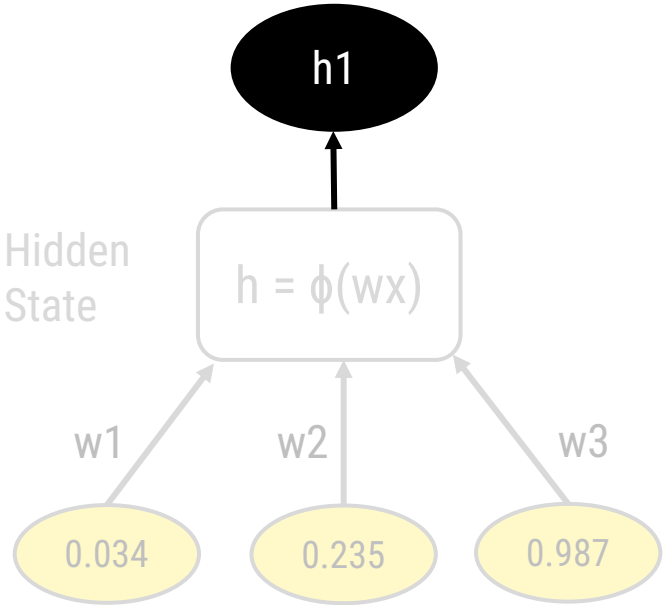
	Word Embeddings (size 3)	
	great	movie
R1	**[0.034, 0.235, 0.987]**	[0.765, 0.523, 0.895]



Example:

Review 1 – "**Great** movie"

	Word Embeddings (size 3)	
	great	movie
R1	**[0.034, 0.235, 0.987]**	[0.765, 0.523, 0.895]

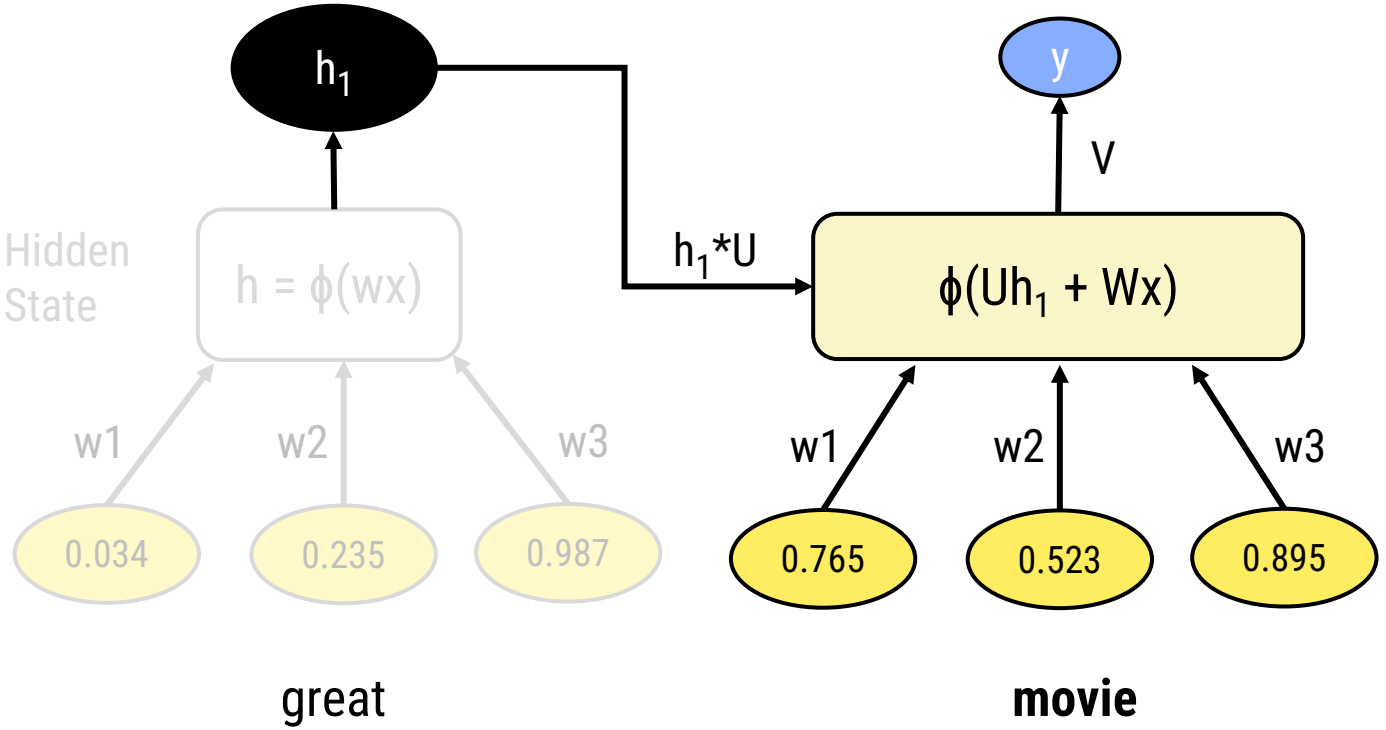


great

Example:

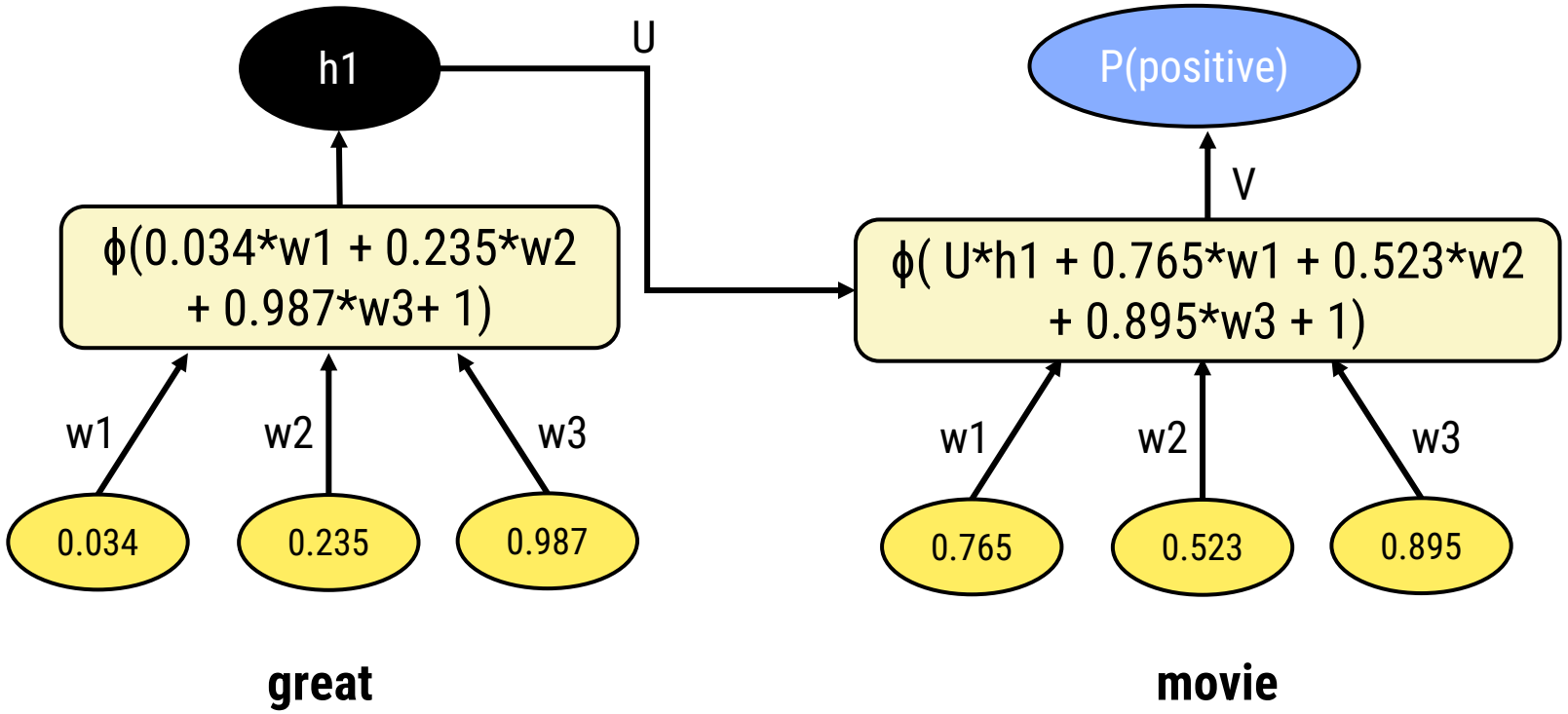
Review 1 – "Great **movie**"

	Word Embeddings (size 3)	
	great	**movie**
R1	[0.034, 0.235, 0.987]	**[0.765, 0.523, 0.895]**



Example:

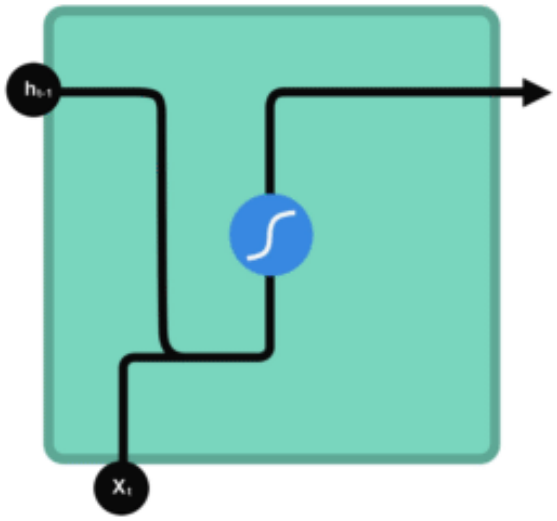
Review 1 – “Great movie”

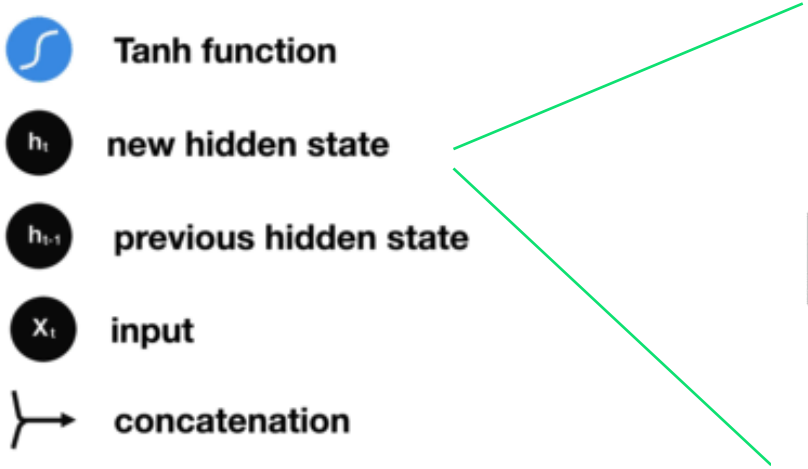


The Architecture behind RNNs

Recurrent Neural Networks

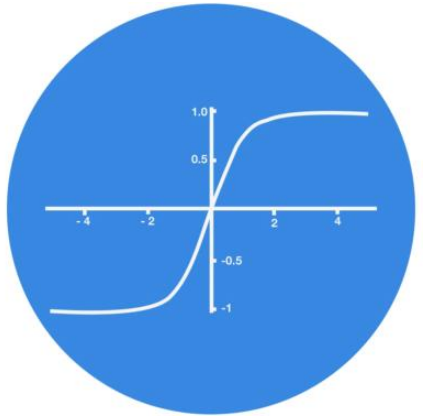
For each word:





5
0.1
-0.5

Tanh

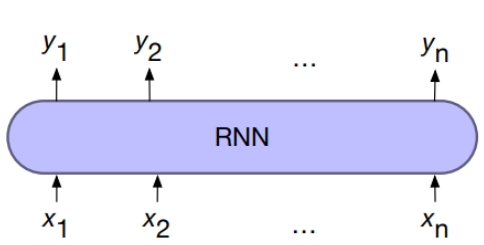


A tanh function ensures that the values stay between -1 and 1, thus regulating the output of the neural network, i.e., allowing the values to stay between the boundaries -1 and 1. Otherwise, some values can explode and become astronomical, causing other values to seem insignificant.

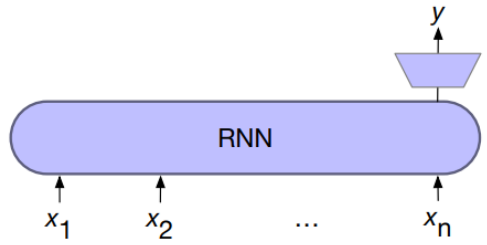
To process sequential input, we can use some well know sequential models, like **Recurrent Neural Networks (RNN)** and **Long Short Term Memory Neural Networks (LSTM)**

Common Tasks

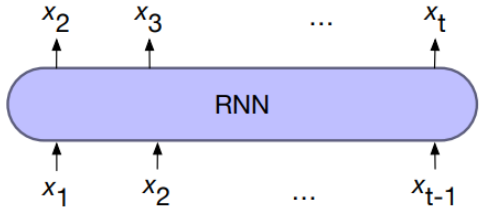
- Next word prediction
- Sequence Labeling
- Sequence Classification
- **Machine Translation**
- **Chatbots**



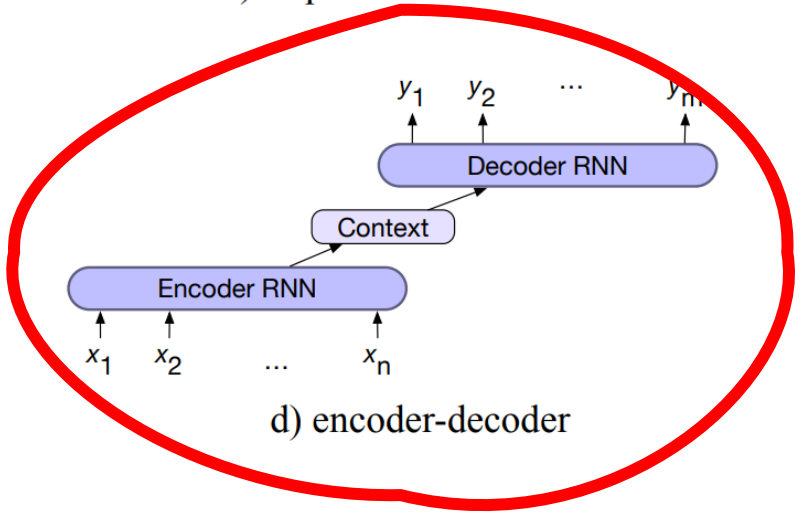
a) sequence labeling



b) sequence classification



c) language modeling



d) encoder-decoder

Example:

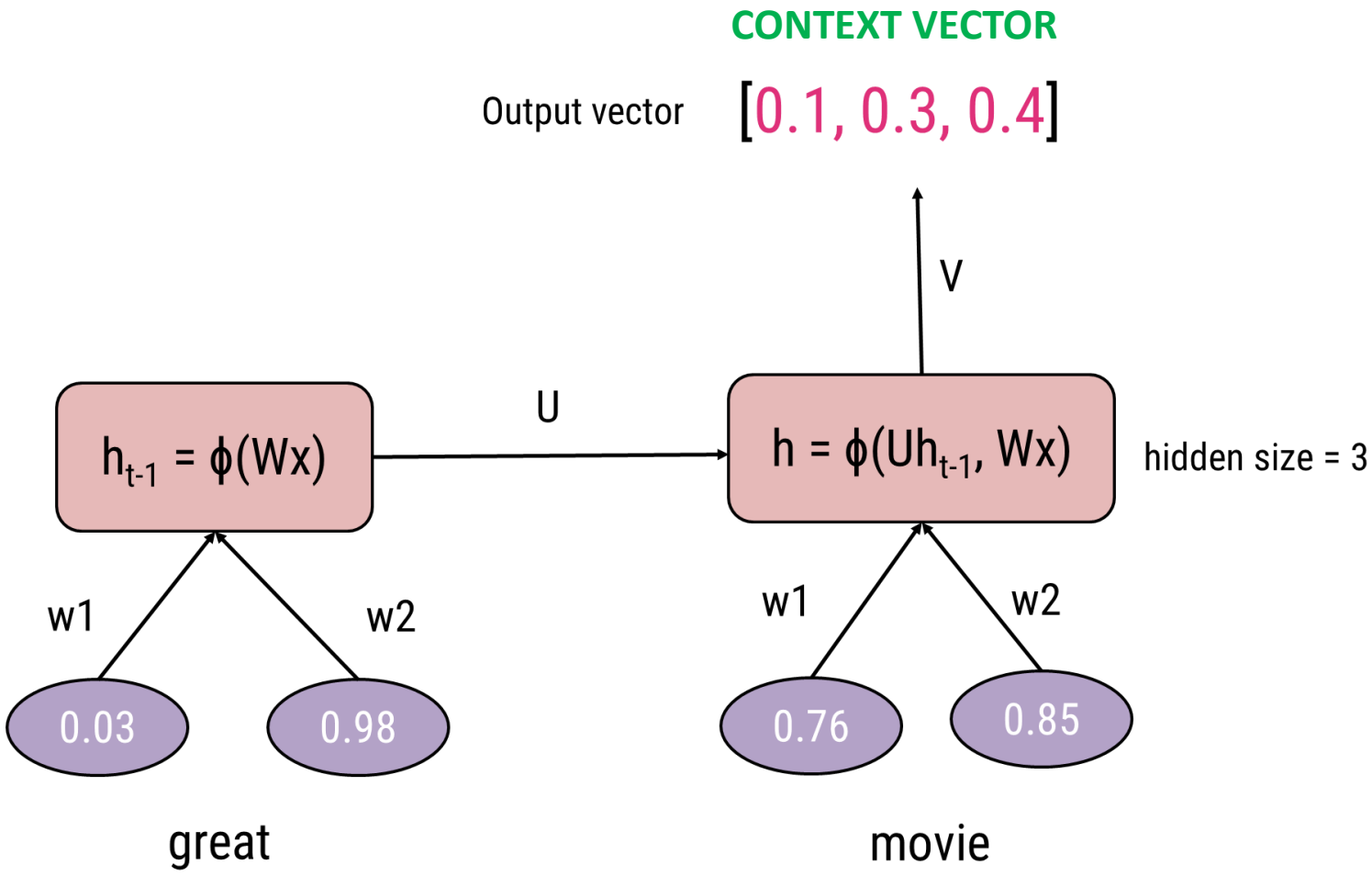
Sentence – “Great movie”

Translation – “Ótimo filme”

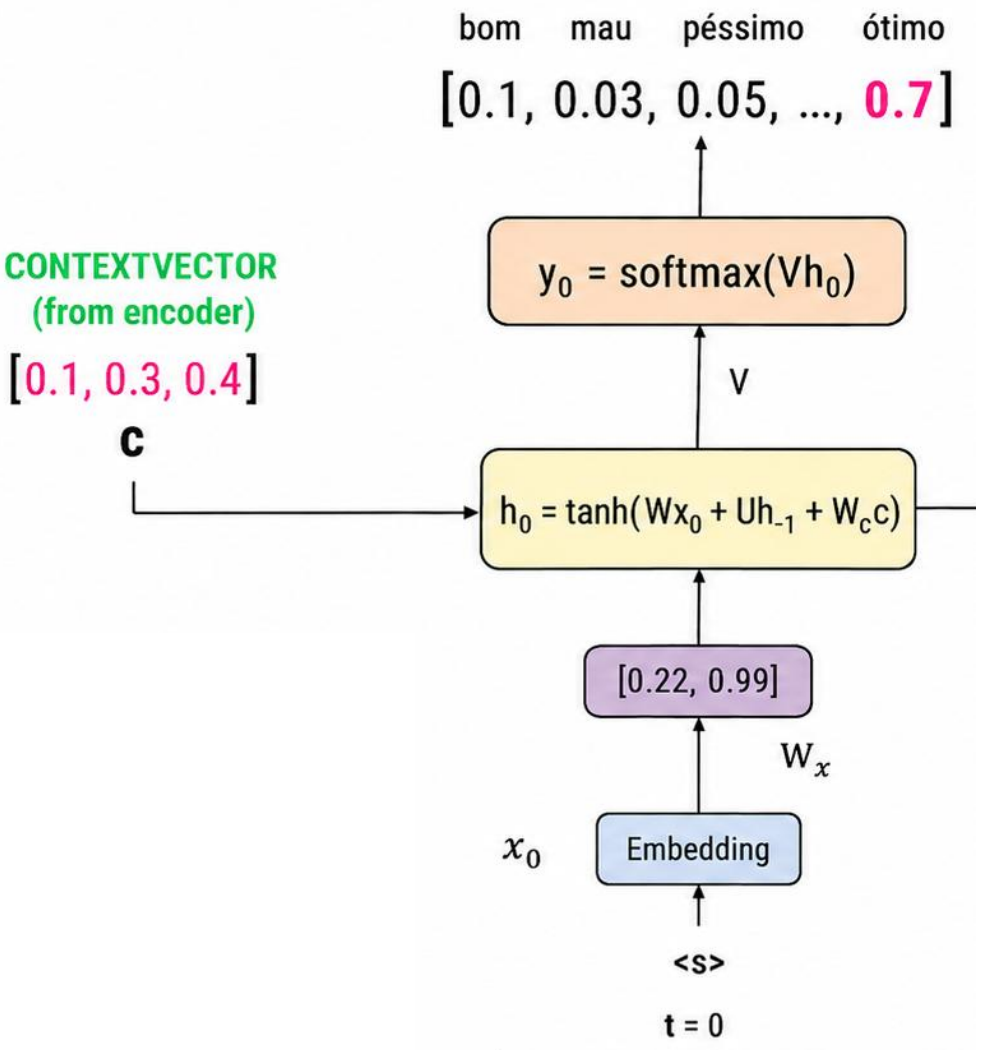
	Word Embeddings (size 2)	
	great	movie
Sentence	[0.03, 0.98]	[0.76, 0.85]

	Word Embeddings (size 2)			
	<s>	ótimo	filme	</s>
Translation	[0.22, 0.99]	[0.40, 0.23]	[0.05, 0.33]	[0.01, 0.76]

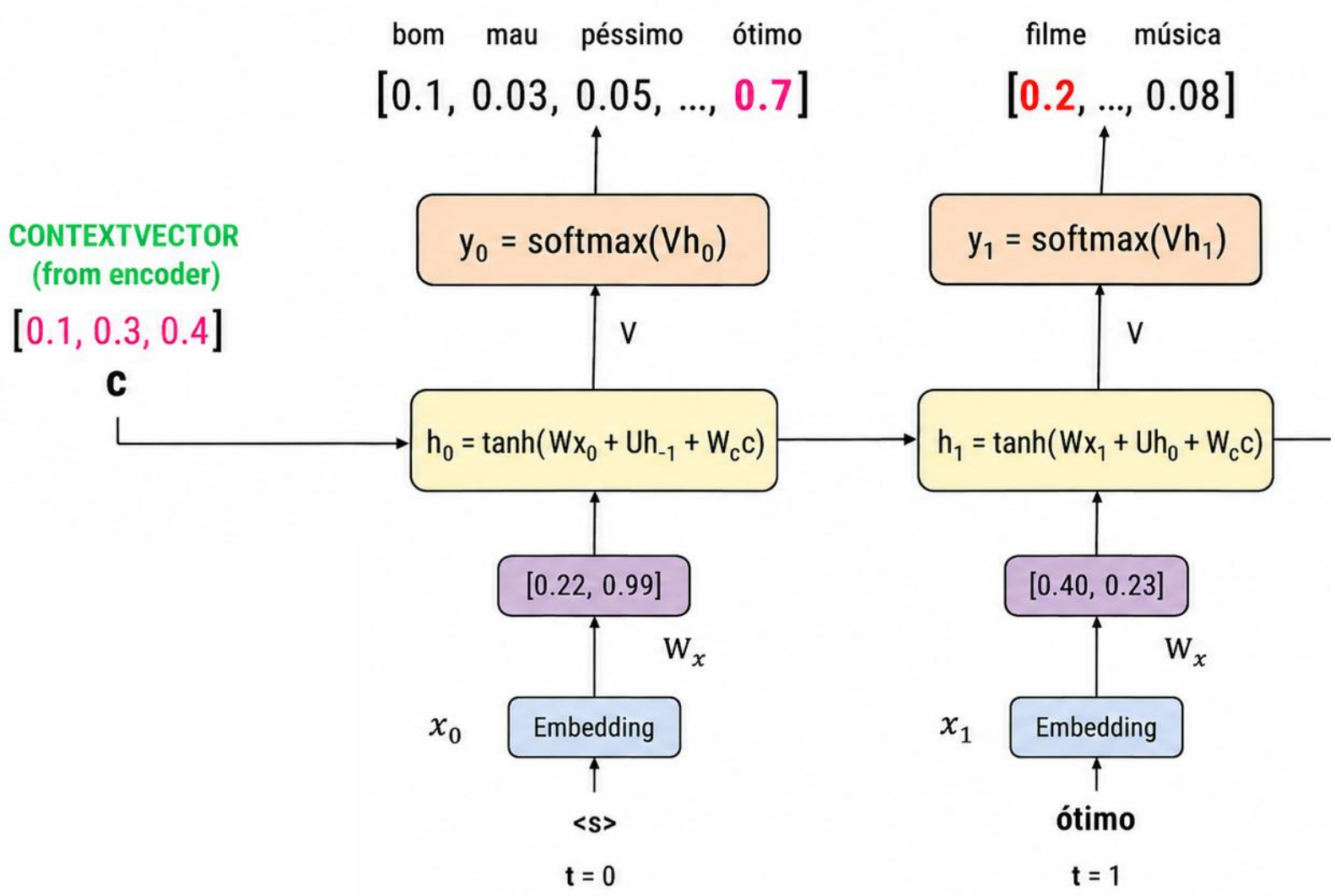
Encoder


CONTEXT VECTOR
Output vector
[0.1, 0.3, 0.4]
V
U
$h_{t-1} = \phi(Wx)$
$h = \phi(Uh_{t-1}, Wx)$
hidden size = 3
w1
w2
w1
w2
0.03
0.98
0.76
0.85
great
movie

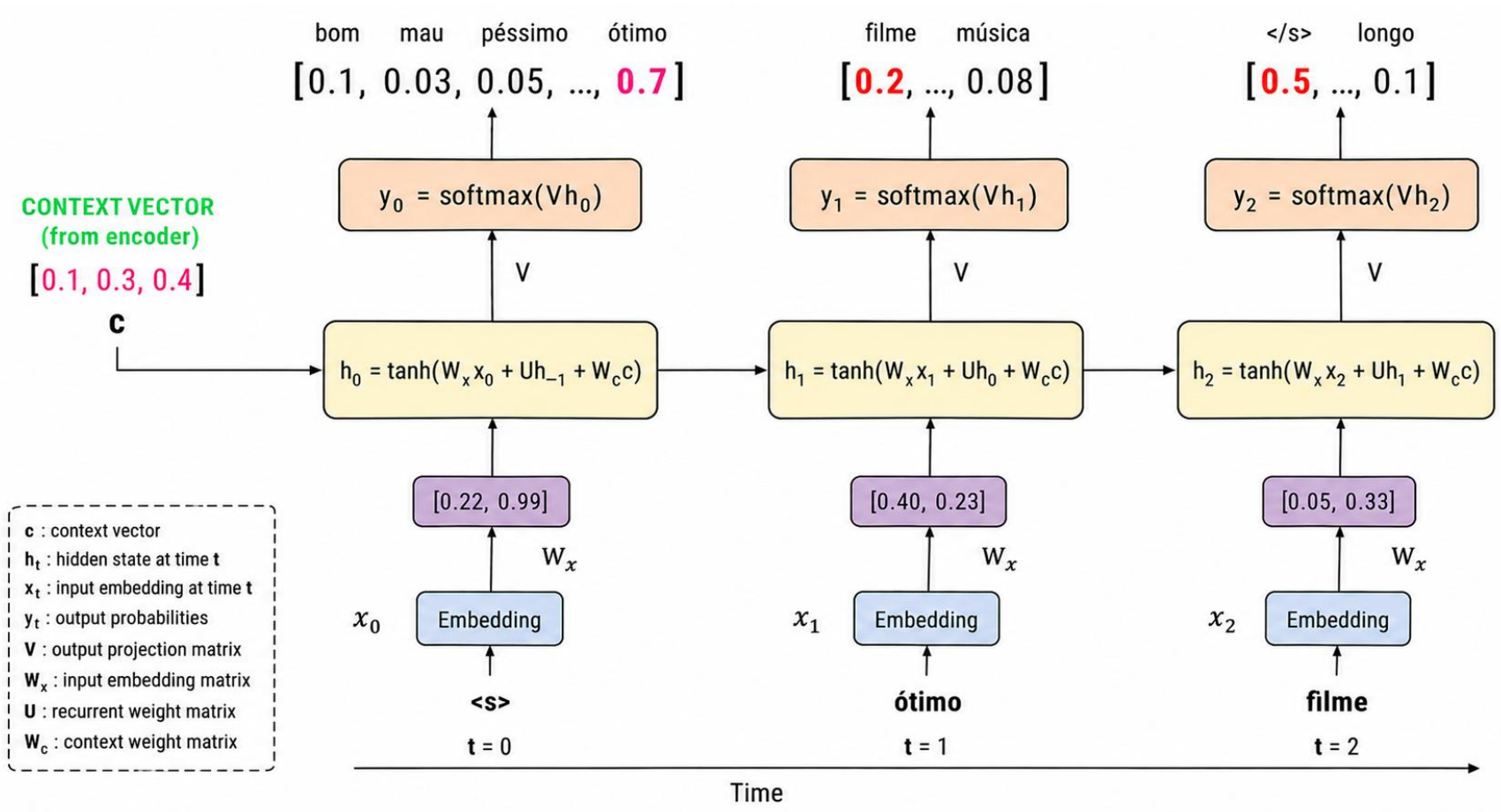
Decoder


bom
mau
péssimo
ótimo
[0.1, 0.03, 0.05, ..., 0.7]
CONTEXTVECTOR
(from encoder)
[0.1, 0.3, 0.4]
c
$y_0 = \text{softmax}(Vh_0)$
V
$h_0 = \tanh(Wx_0 + Uh_{-1} + W_c c)$
[0.22, 0.99]
W_x
x_0
Embedding
<s>
t = 0

Decoder


bom
mau
péssimo
ótimo
[0.1, 0.03, 0.05, ..., 0.7]
filme
música
[0.2, ..., 0.08]
CONTEXTVECTOR
(from encoder)
[0.1, 0.3, 0.4]
c
$y_0 = \text{softmax}(Vh_0)$
$y_1 = \text{softmax}(Vh_1)$
V
V
$h_0 = \tanh(Wx_0 + Uh_{-1} + W_c c)$
$h_1 = \tanh(Wx_1 + Uh_0 + W_c c)$
[0.22, 0.99]
[0.40, 0.23]
W_x
W_x
x_0
x_1
Embedding
Embedding
<s>
ótimo
t = 0
t = 1

Decoder



After predicting </s> the network stops or moves to the next sentence.

02.

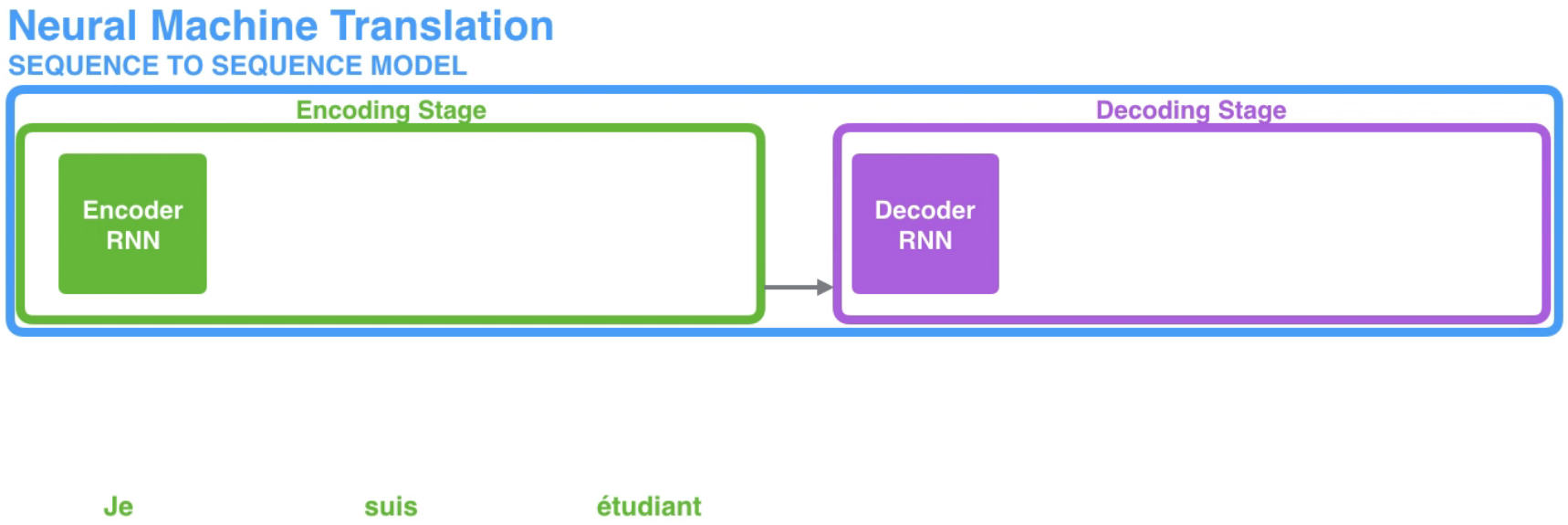
Attention Mechanism

The **decoder is supposed to generate a translation solely based on the last hidden state from the encoder**. This **last hidden state vector must encode everything** we need to know about the source sentence. It must fully capture its meaning. In more technical terms, that **vector is a sentence embedding**.

In summary, **classical encoder-decoder RNN have to encode all information about a sentence into a single vector** and then the **decoder must produce a translation based on only that vector**.

Let's say your source sentence is 50 words long. The first word of the English translation is probably highly correlated with the first word of the source sentence. But that means the decoder has to consider information from 50 steps ago, and that information needs to be somehow encoded in the vector 😨 .

In summary, **classical encoder-decoder RNN have to encode all information about a sentence into a single vector** and then the **decoder must produce a translation based on only that vector**.



https://jalammar.github.io/images/seq2seq_6.mp4

This **final hidden state from the encoder is acting as a bottleneck**: it must represent absolutely everything about the meaning of the source text, since the only thing **the decoder knows about the source text is what's in this context vector**.

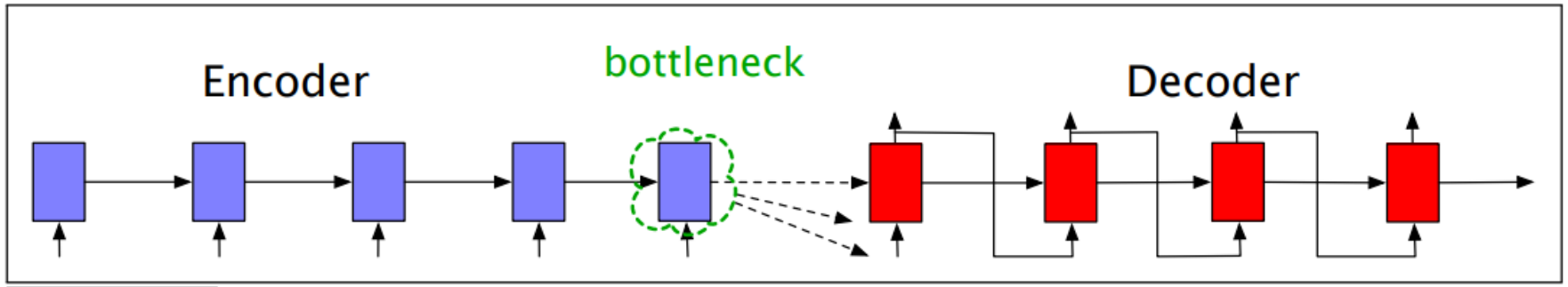


Figure 9.21 Requiring the context c to be only the encoder's final hidden state forces all the information from the entire source sentence to pass through this representational bottleneck.

Jurafsky, D., & Martin, J. H. (n.d.). **Speech and language processing** (3rd ed. draft). May 7, 2026, from https://web.stanford.edu/~jurafsky/slp3/

How Models Learn What to Focus On

Attention Mechanism

Languages structure information differently; **important semantic information does not appear at the same point in a sentence across languages**.

In some languages, such as English, the verb appears early, allowing listeners or models to quickly understand the action being performed. In others, such as Korean or certain Dutch constructions, the verb appears at the end of the sentence, forcing the listener or model to maintain information in memory and delay interpretation until later.

How Models Learn What to Focus On

Attention Mechanism

An alternative approach is to use a mechanism called **Attention**.

How Models Learn What to Focus On

Attention Mechanism

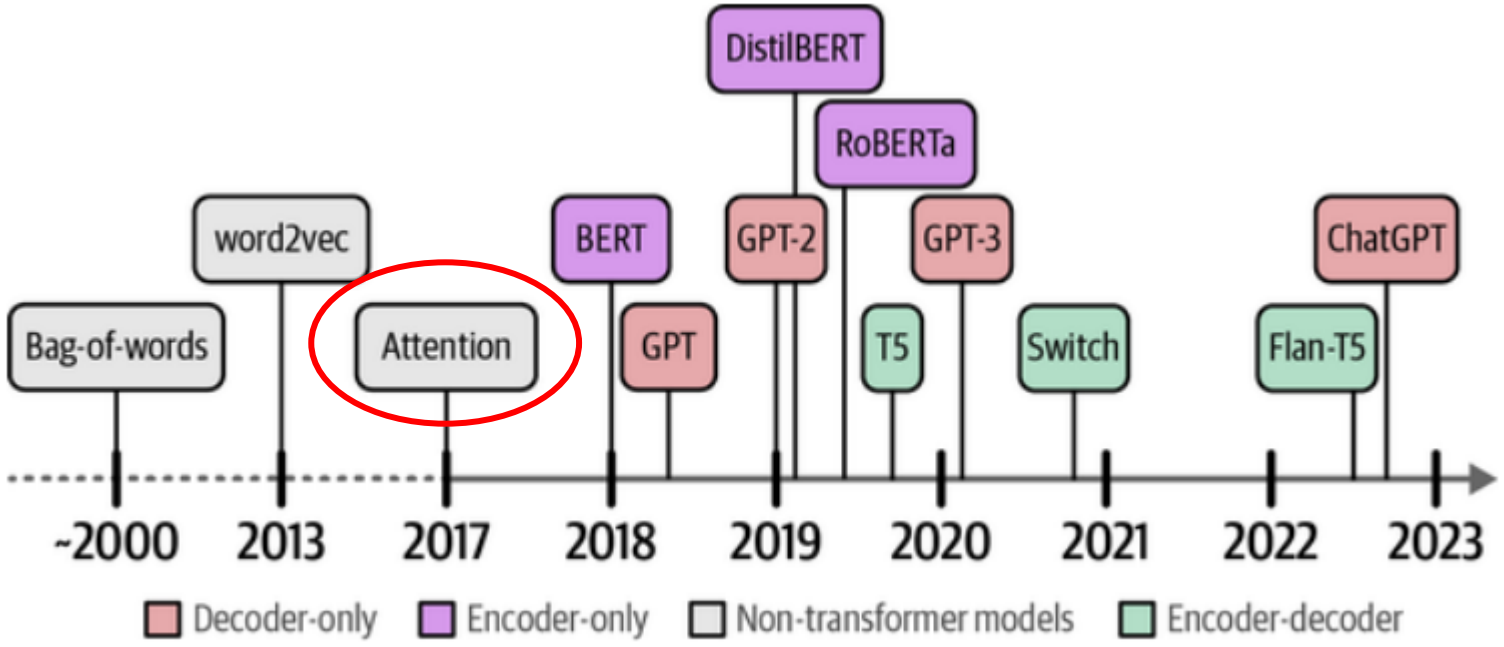


Figure 1-1. A peek into the history of Language AI.

Alammar, J., & Grootendorst, M. (2024). **Hands-On large language models: Language Understanding and Generation**. Sebastopol : O'Reilly Media.

How Models Learn What to Focus On

Attention Mechanism

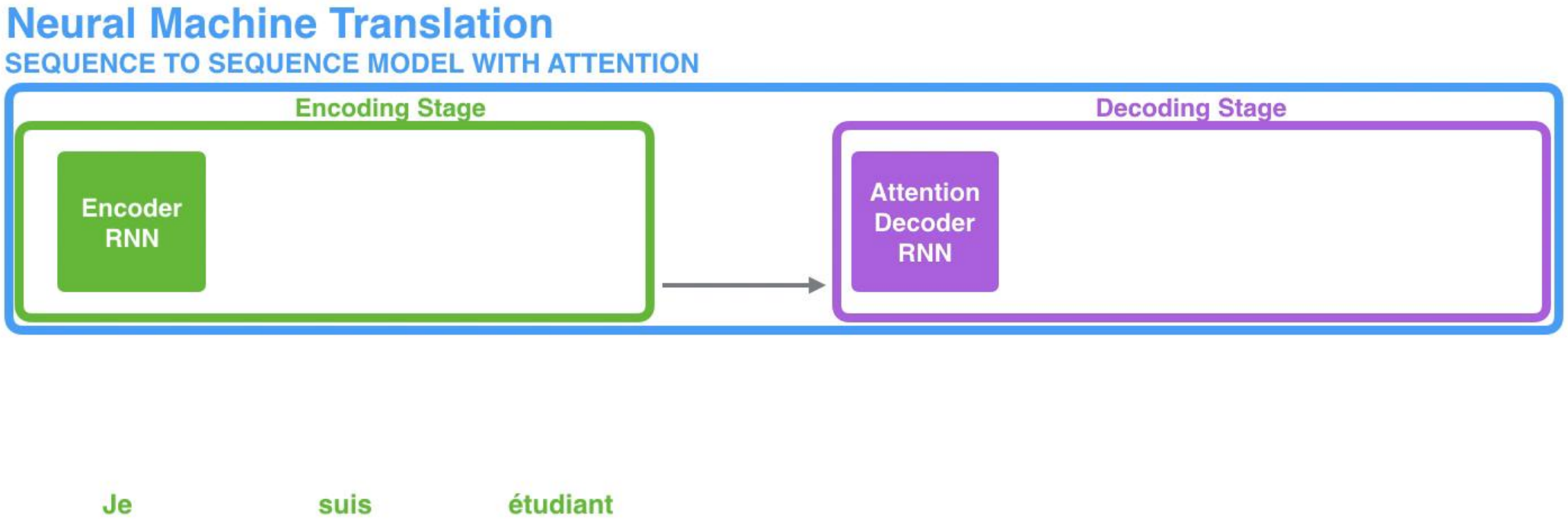
The **Attention mechanism** (Bahdanau et al., 2014 and Luong et al., 2015) is a solution to the bottleneck problem, **allowing the decoder to get information from all the hidden states of the encoder**, not just the last hidden state.

The idea of attention is to create the a **fixed-length vector** by taking a **weighted sum of all the encoder hidden states**.

Attention thus **replaces the static vector** with one that is **dynamically derived from the encoder hidden states**, different for each token in decoding.

How Models Learn What to Focus On

Attention Mechanism



https://jalammar.github.io/images/seq2seq_7.mp4

How Models Learn What to Focus On

Attention Mechanism

At each step, **the decoder does an extra step before producing its output**.
In order to focus on the parts of the input that are relevant to this decoding time step, the decoder does the following:

1. Look at the set of encoder hidden states it received – **each encoder hidden state is most associated with a certain word in the input sentence**

2. Give **each hidden state a score** (let's ignore how the scoring is done for now)

3. Multiply each hidden state by its **softmaxed score**, thus amplifying hidden states with high scores, and drowning out hidden states with low scores

How Models Learn What to Focus On

Attention Mechanism

https://jalammar.github.io/images/attention_process.mp4

But how to compute this dynamic context for each decoder time step?

STEP 1

The first step is to **compute how much to focus on each encoder state**.

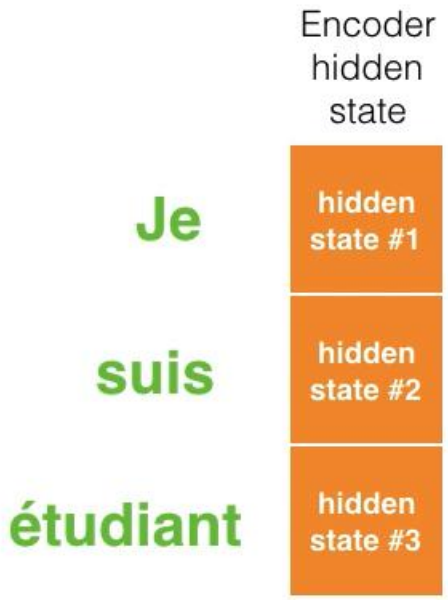
We capture relevance by computing - at each state i during decoding – a score measure – for each encoder state j.

This score is called **dot product attention**:

$$\text{score}(\mathbf{h}_{i-1}^{d}, \mathbf{h}_{j}^{e}) = \mathbf{h}_{i-1}^{d} \cdot \mathbf{h}_{j}^{e}$$

How Models Learn What to Focus On

Attention Mechanism



https://jalammar.github.io/images/seq2seq_9.mp4

But how to compute this dynamic context for each decoder time step?

STEP 2

The score that results from this dot product is **a scalar that reflects the degree of similarity** between the two vectors.

To make use of these scores, we'll **normalize them with a softmax to create a vector of weights**, α_{ij}, that tells us the proportional relevance of each encoder hidden state *j* to the prior hidden decoder state h^d_{i-1}

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{h}^d_{i-1}, \mathbf{h}^e_j) \ \forall j \in e)$$

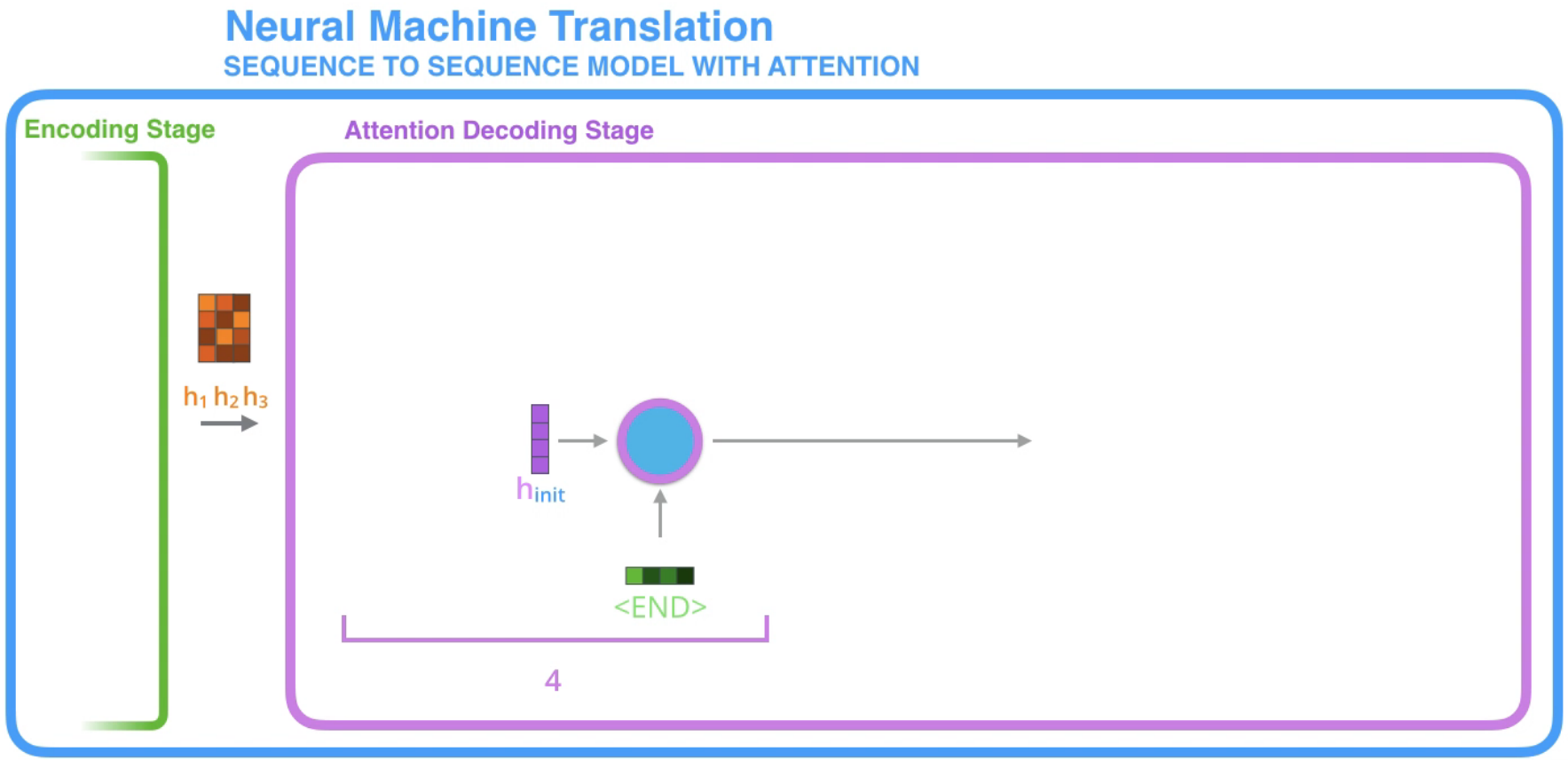
But how to compute this dynamic context for each decoder time step?

Finally, given the distribution in α, we can **compute a fixed-length context vector for the current decoder state** by taking a **weighted average** over all the encoder hidden states.

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$

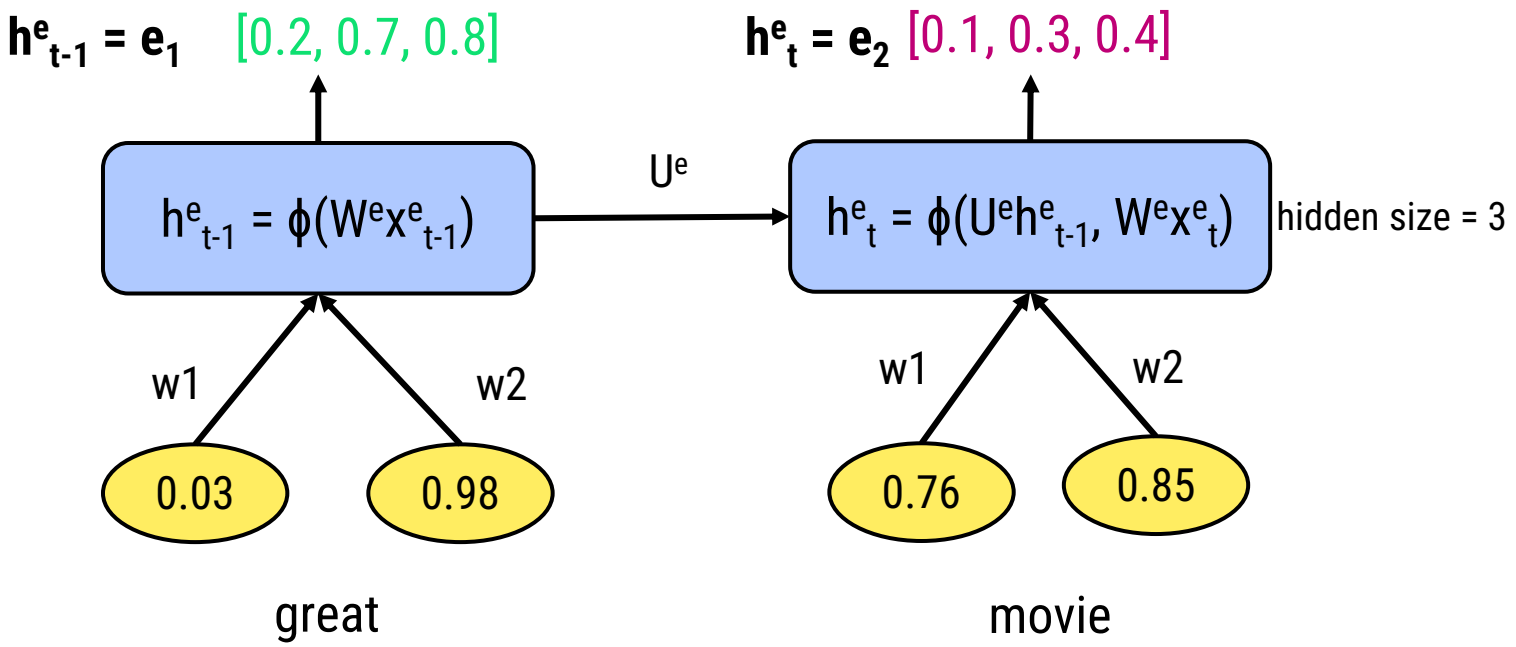
How Models Learn What to Focus On

Attention Mechanism



https://jalammar.github.io/images/attention_tensor_dance.mp4

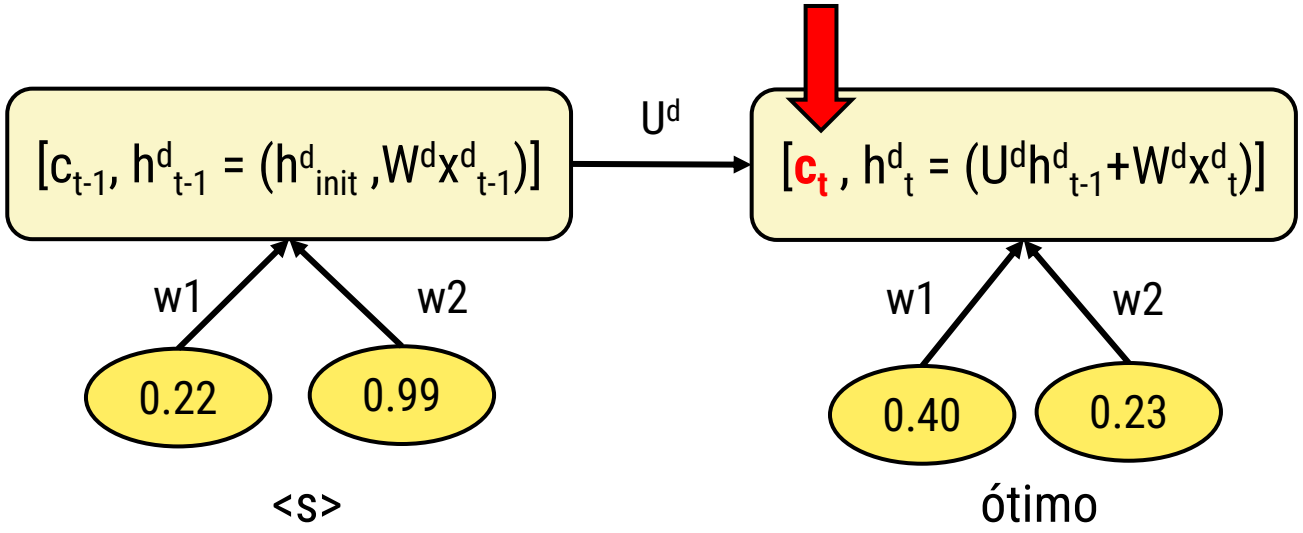
Encoder


$h^e_{t-1} = e_1$ [0.2, 0.7, 0.8]
$h^e_t = e_2$ [0.1, 0.3, 0.4]
$h^e_{t-1} = \phi(W^e x^e_{t-1})$
U^e
$h^e_t = \phi(U^e h^e_{t-1}, W^e x^e_t)$
hidden size = 3
w1
w2
w1
w2
0.03
0.98
0.76
0.85
great
movie

Decoder

What word to translate next?

We need to learn the context vector



Decoder

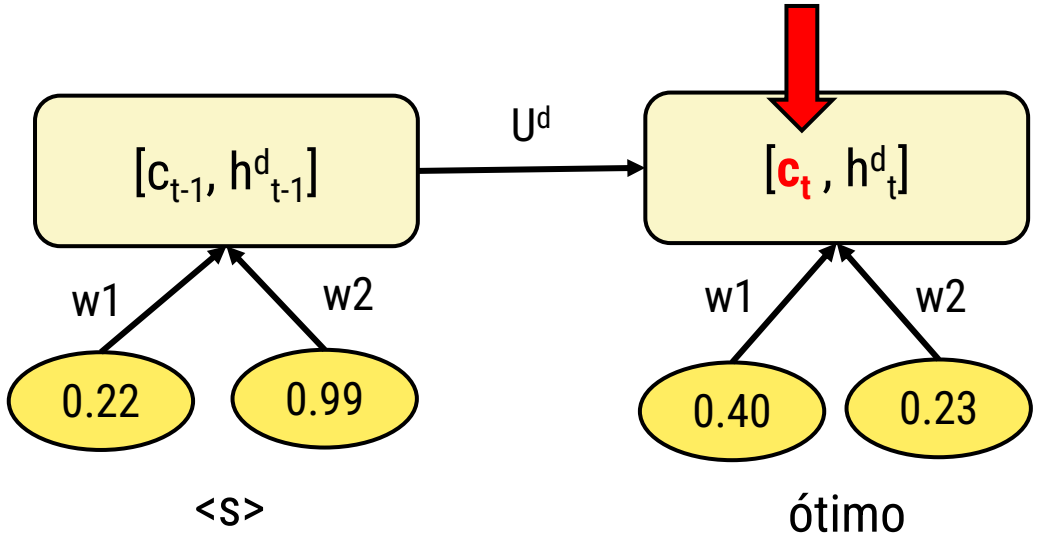
Recall the Encoder's Hidden States

e1 [0.2, 0.7, 0.8]

e2 [0.1, 0.3, 0.4]

What word to translate next?

We need to learn the context vector

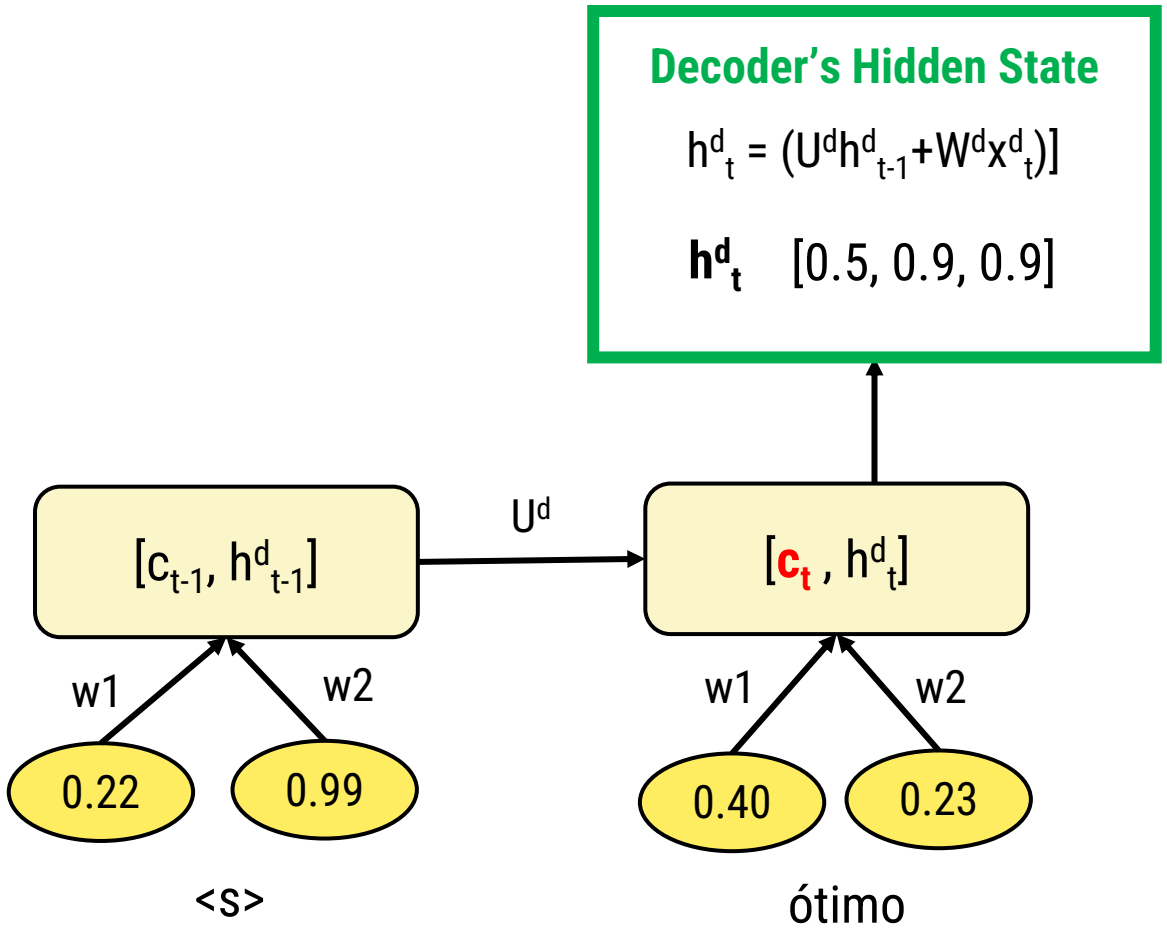


Decoder

Encoder's Hidden States

e1 [0.2, 0.7, 0.8]

e2 [0.1, 0.3, 0.4]



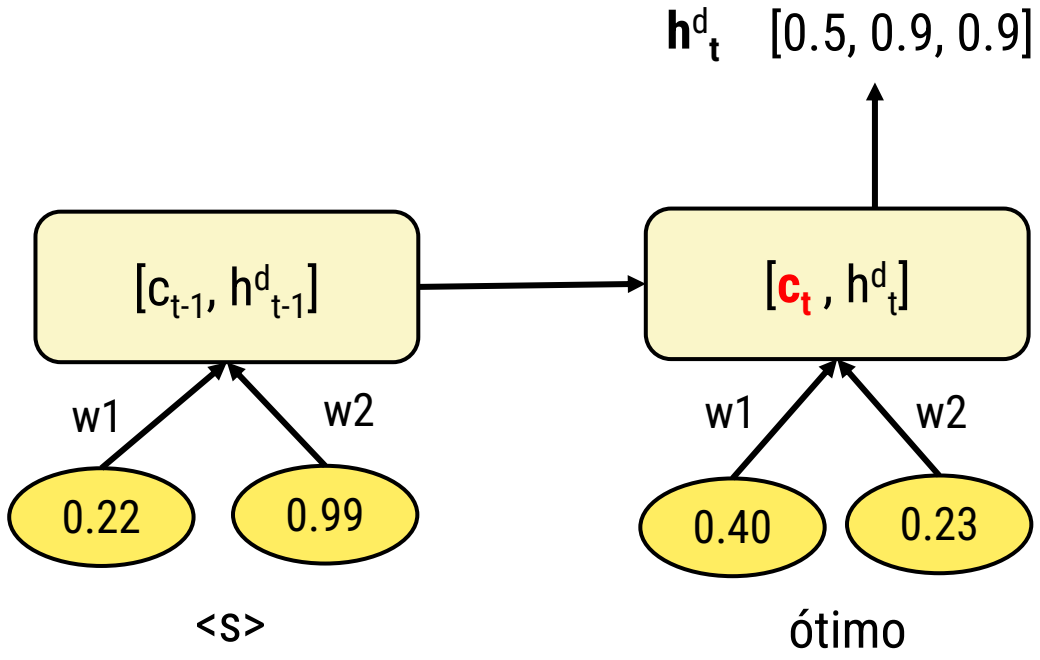
Decoder

Attention Score (step 1)

Dot product between encoder hidden states and decoder hidden state

e1 [0.2, 0.7, 0.8] • h^d_{t-1} [0.5, 0.9, 0.9]
= 0.1+0.63+0.72 = **1.45**

e2 [0.1, 0.3, 0.4] • h^d_{t-1} [0.5, 0.9, 0.9]
= **1.13**

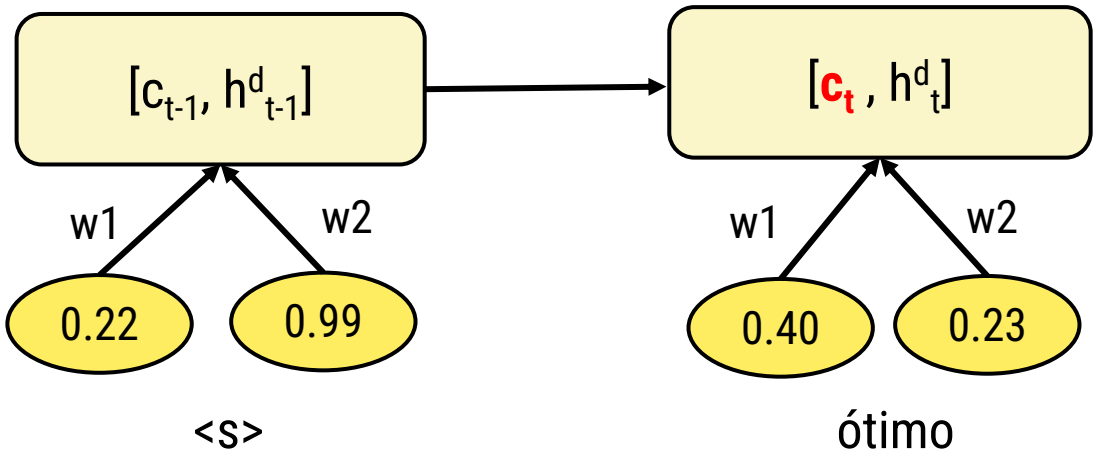


Decoder

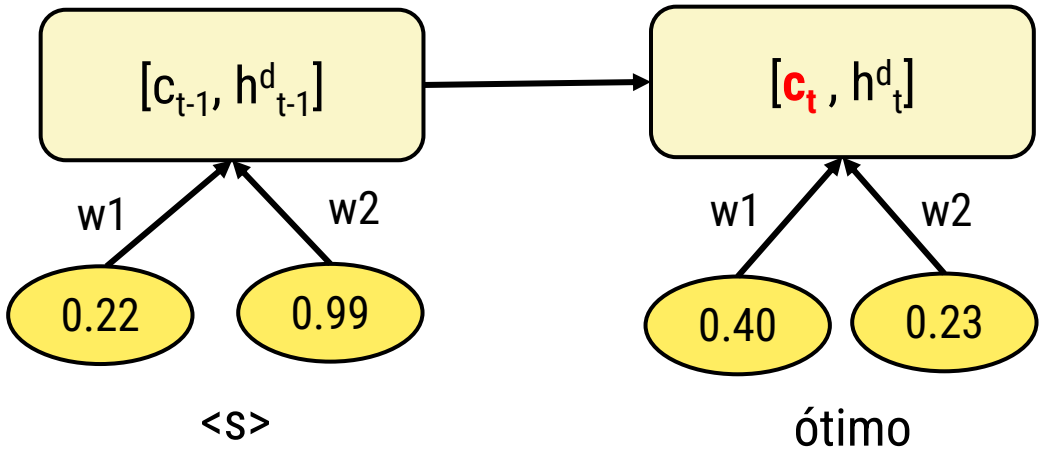
Attention Score (step 2)

Normalize scores with a softmax to create a vector of weights

Softmax([1.45, 1.13]) = **[0.58, 0.42]**



Decoder



Attention Score (step 3)

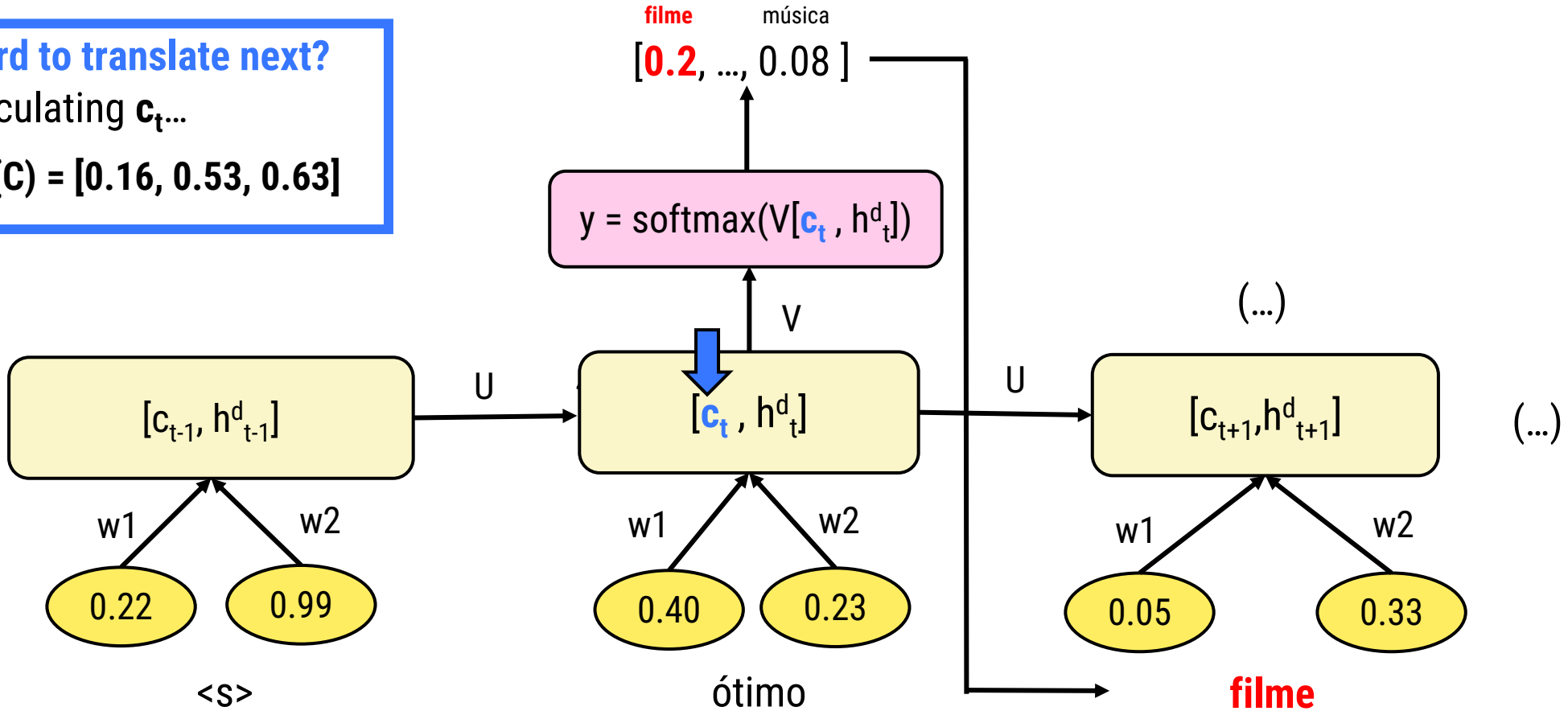
Compute a fixed-length context vector for the current decoder state by taking a **weighted average over all the encoder hidden states**.

Context vector (c_t) = (**0.58***[0.2, 0.7, 0.8]) + (**0.42***[0.1, 0.3, 0.4]) = [0.12, 0.41, 0.46] + [0.04, 0.13, 0.17] = **[0.16, 0.53, 0.63]**

Decoder

What word to translate next?
After calculating $\mathbf{c_t}$...

Context (C) = [0.16, 0.53, 0.63]



03. Transformer Architecture

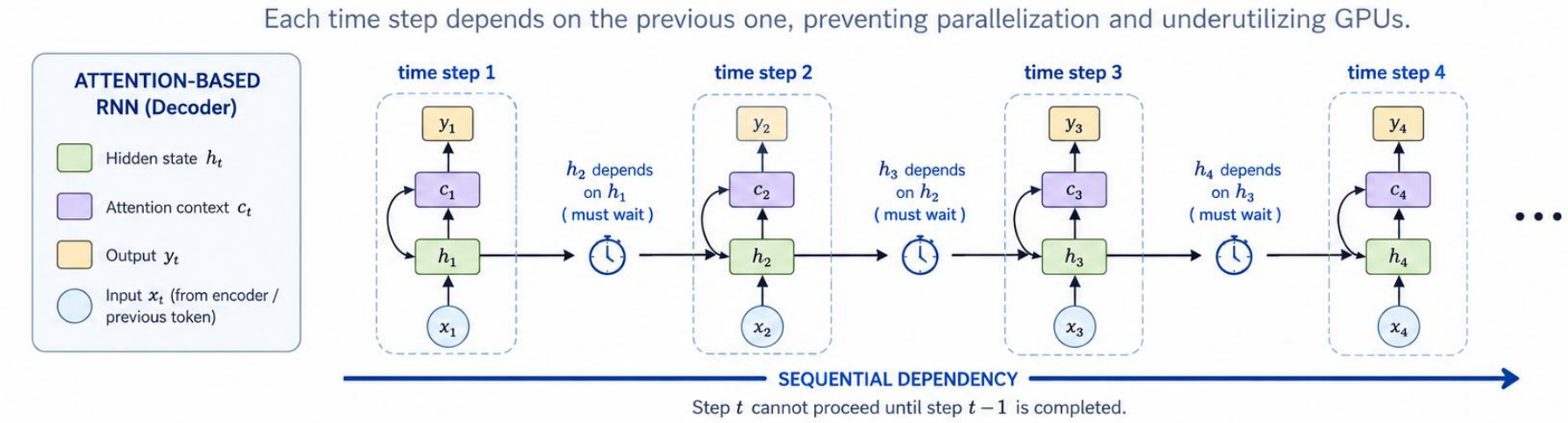
Deep learning increasingly relied on **GPUs** (Graphics Processing Unit) because they can perform many computations in parallel.[1]

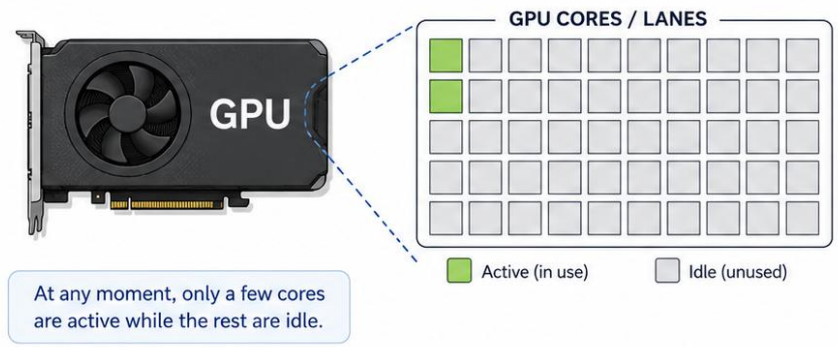
However, **RNN-based language models process tokens one word at a time**, with each hidden state depending on the previous one, so **computation must proceed sequentially**.

This can be **very time-consuming and limits parallel processing,** meaning GPUs cannot fully use their many cores during training.

[1] https://en.wikipedia.org/wiki/Graphics_processing_unit

The **sequential nature of attention based RNNs prevents parallelization during training of the model,** making this process **less efficient and not well-optimized for GPUs**





To solve this issue **Self-attention** was formally introduced in the landmark 2017 paper titled **"Attention Is All You Need" by Vaswani et al. (2017)**

Self-attention **allows each token to directly attend to all other tokens in the sequence at the same time**, capturing dependencies regardless of distance

The authors proposed a new model called **"Transformer"**, that uses Self-attention and Feed-forward Neural Networks **eliminating sequential computation**, and enabling fast and efficient training on modern hardware

The origin of Transformers

Transformer Architecture

The origin of Transformers

Transformer Architecture

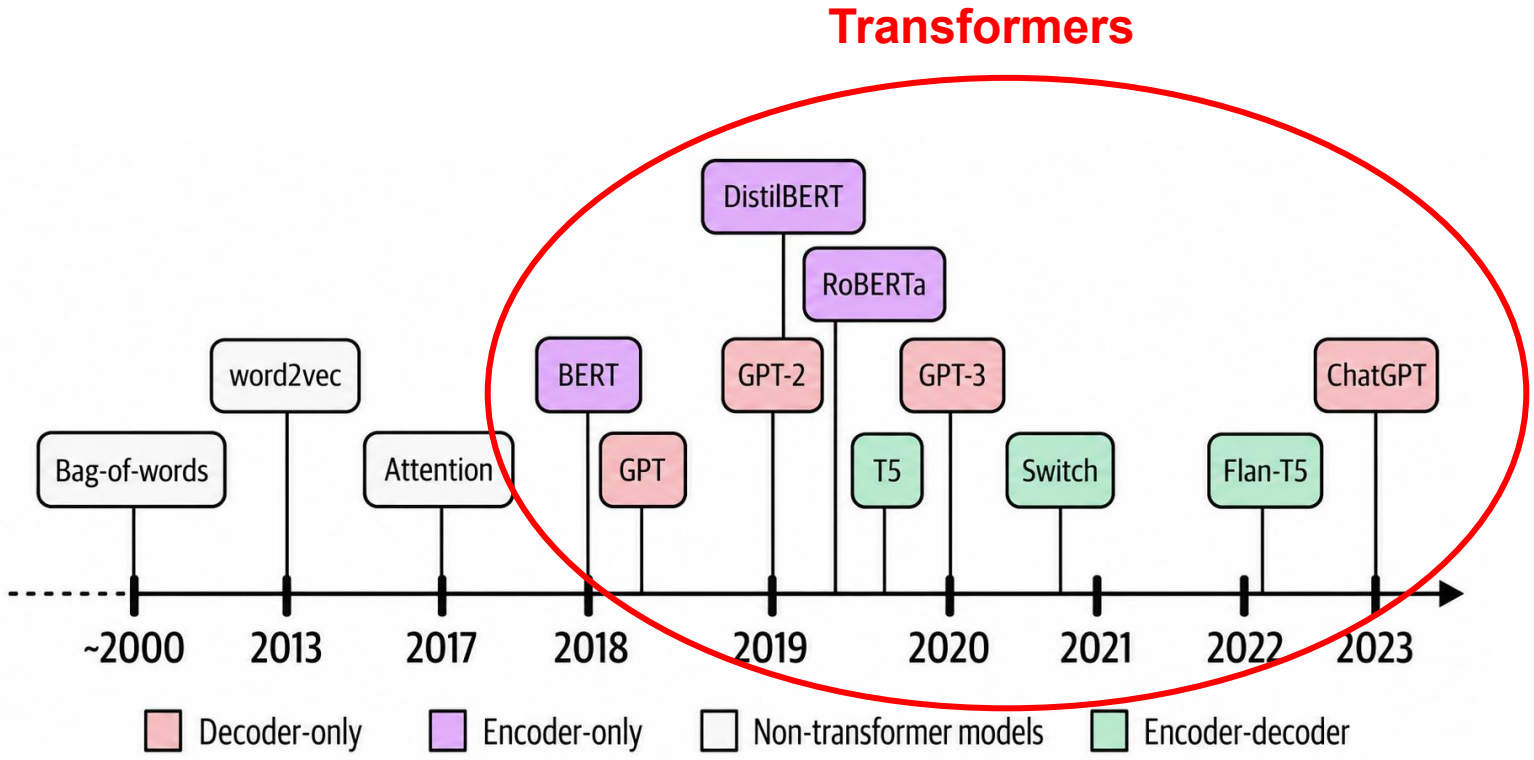


Figure 1-1. A peek into the history of Language AI.

Alammar, J., & Grootendorst, M. (2024). **Hands-On large language models: Language Understanding and Generation**. Sebastopol : O'Reilly Media.

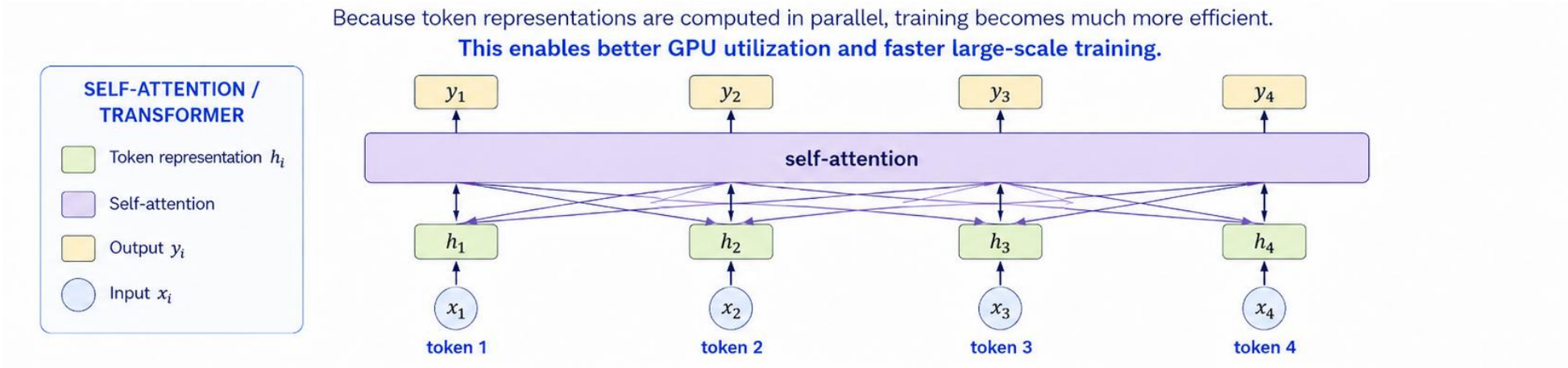
The standard architecture for building modern large language models is the **Transformer**.

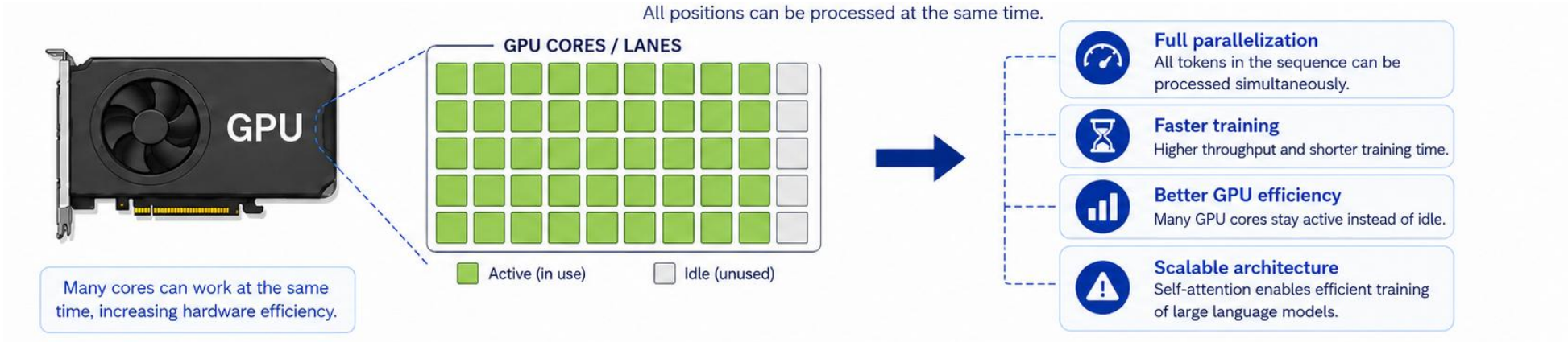
Like LSTMs, Transformers can capture long-distance relationships between words in a sequence. However, **unlike LSTMs, Transformers do not rely on recurrent connections that process tokens step by step**.

Instead, **Transformers use self-attention**, a mechanism that allows each token to directly attend to all other tokens in the sequence. This makes it possible to **compute token representations in parallel**, **improving training efficiency and making much better use of GPUs**.

The origin of Transformers

Transformer Architecture





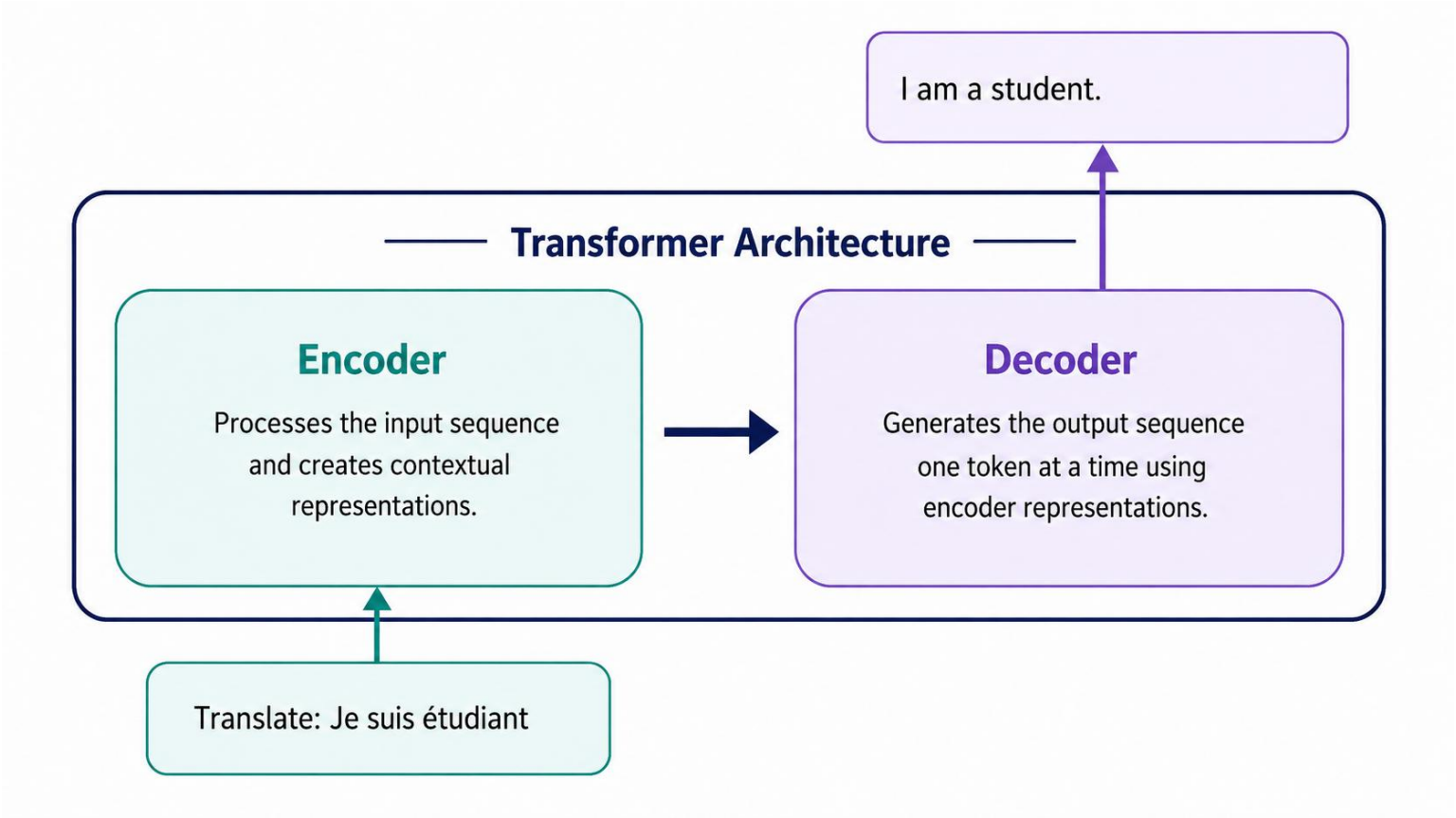
Transformers are made of **two main types of blocks**: encoders and decoders.

The **Encoder** reads the input sequence and transforms **each token into a contextual representation**.

The **Decoder** uses these representations to **generate the output sequence**, one token at a time.

In the original Transformer architecture, the encoder and decoder work together: the encoder represents the input, and the decoder produces the output based on both the previous generated tokens and the encoder's representations.

Transformers are made of **two main types of blocks**: **Encoder** and **Decoder**.

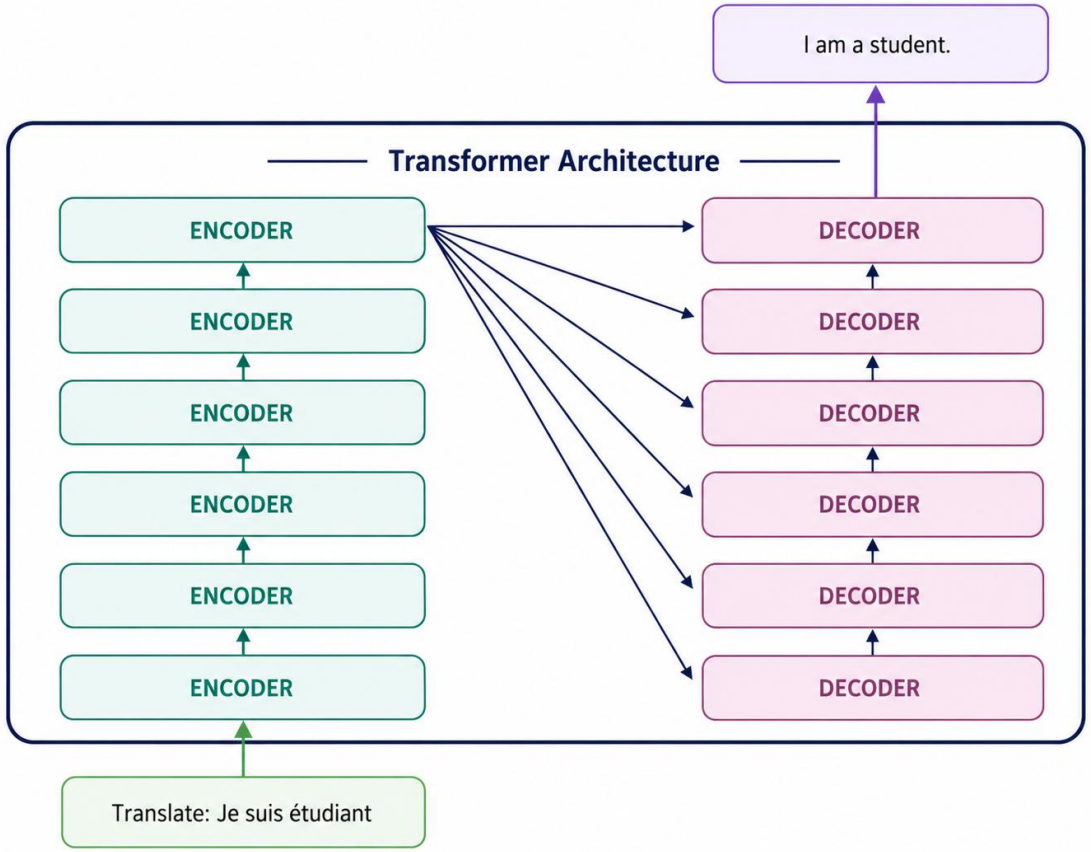


Each transformer will have **many encoders and many decoders**

Inside each **Encoder**, we will find a **layer of self-attention** and a **standard Neural Network** – information moves forward through the encoders

Decoders receive the output from the Encoder and **move them through two attention layers** (self-attention and encoder-decoder attention) and a (feed-forward) **neural network**. The last decoder layer produces an output

Each transformer will have **many encoders and many decoders**

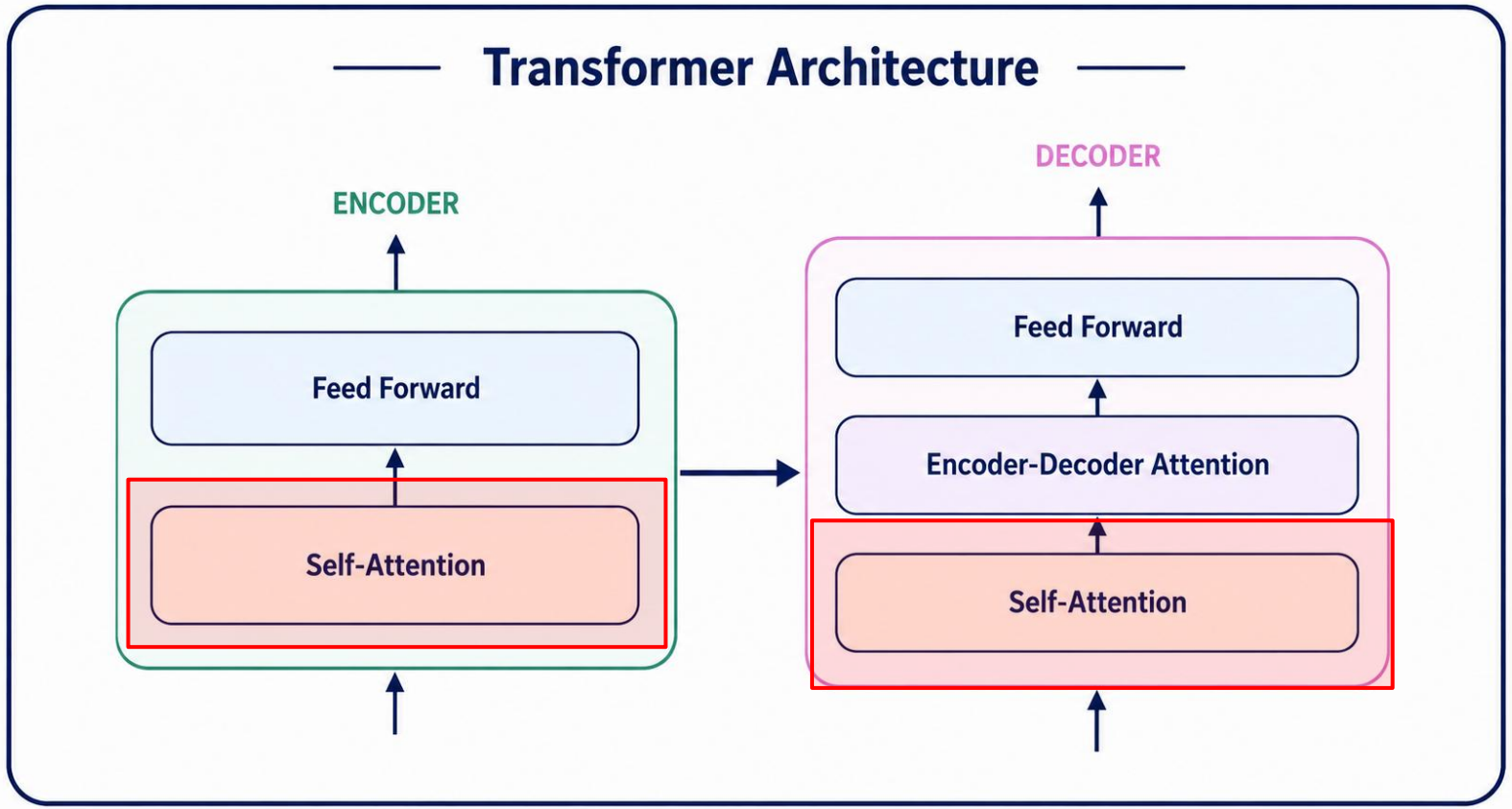


Inside each **Encoder**, we will find a **layer of self-attention** and a **standard Neural Network** – information moves forward through the encoders

How does **Self-Attention work**?

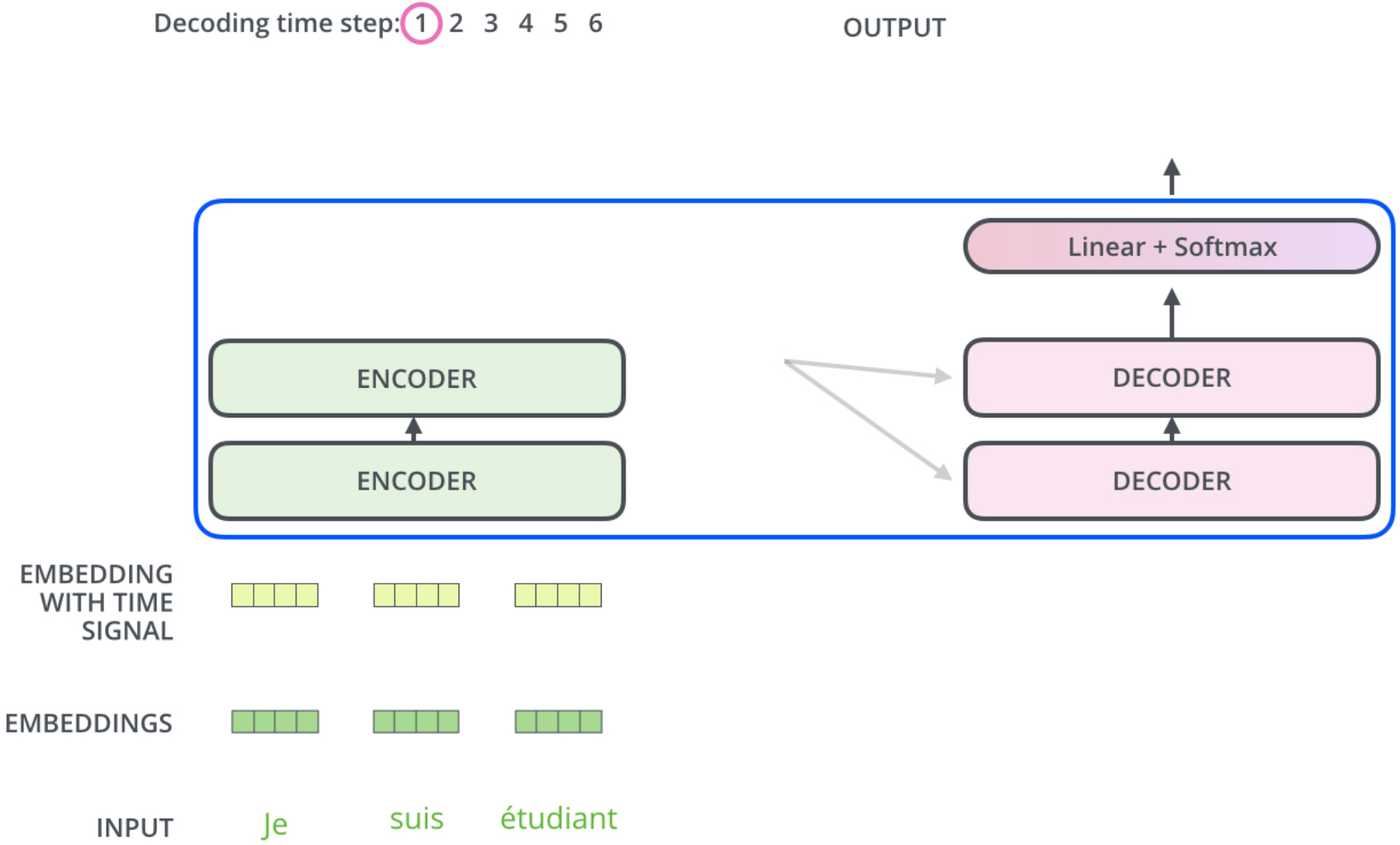
How's it different from the Attention we've seen so far?

Next week, we will focus on the mechanism of **Self-Attention**.



The Architecture behind Transformers

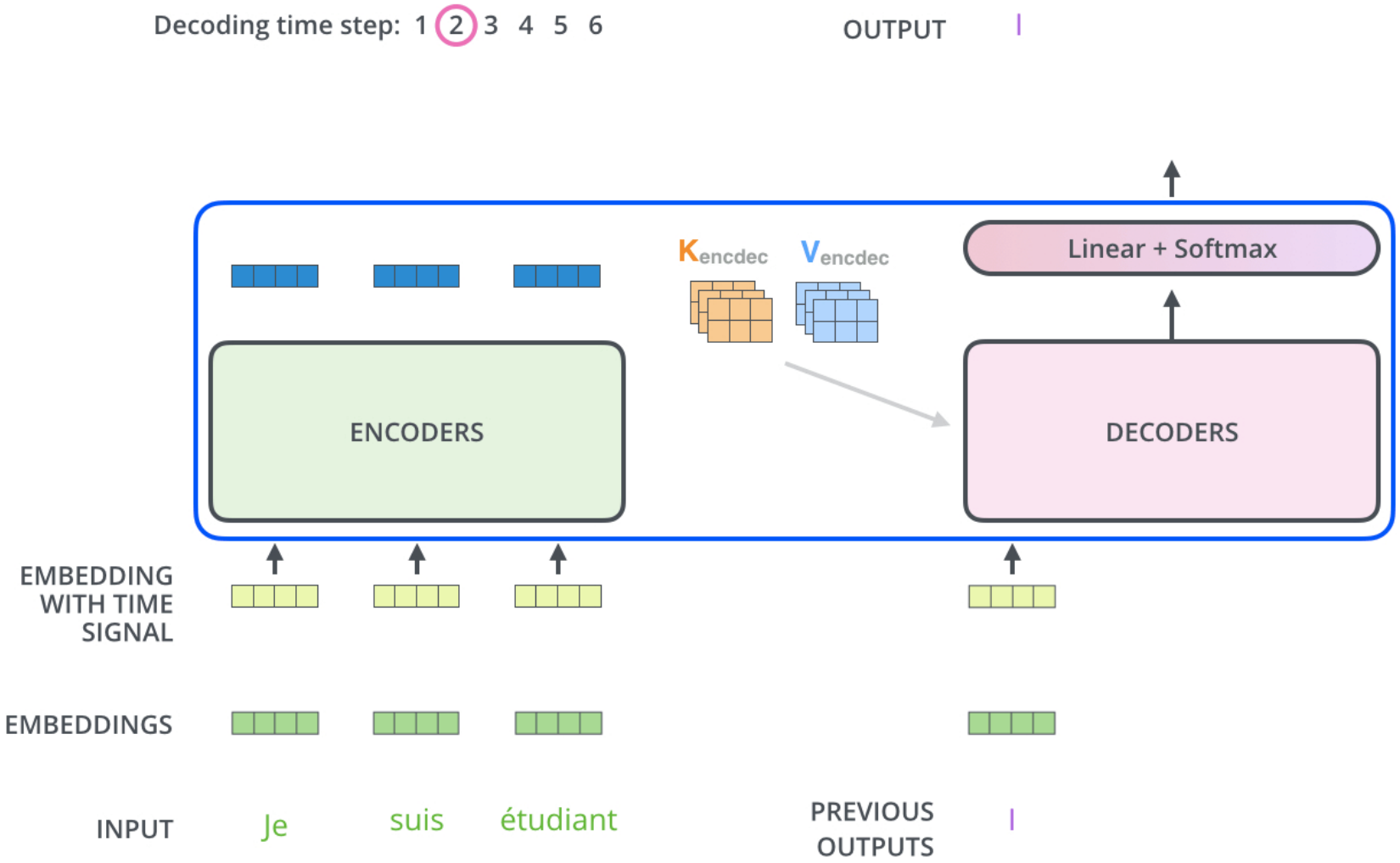
hitecture



https://jalammar.github.io/images/t/transformer_decoding_1.gif

The Architecture behind Transformers

Transformer Architecture



https://jalammar.github.io/images/t/transformer_decoding_2.gif

04.

Transformer Encoders

Some Transformers use **only the encoder**, because the goal is to **understand text rather than generate new text**.

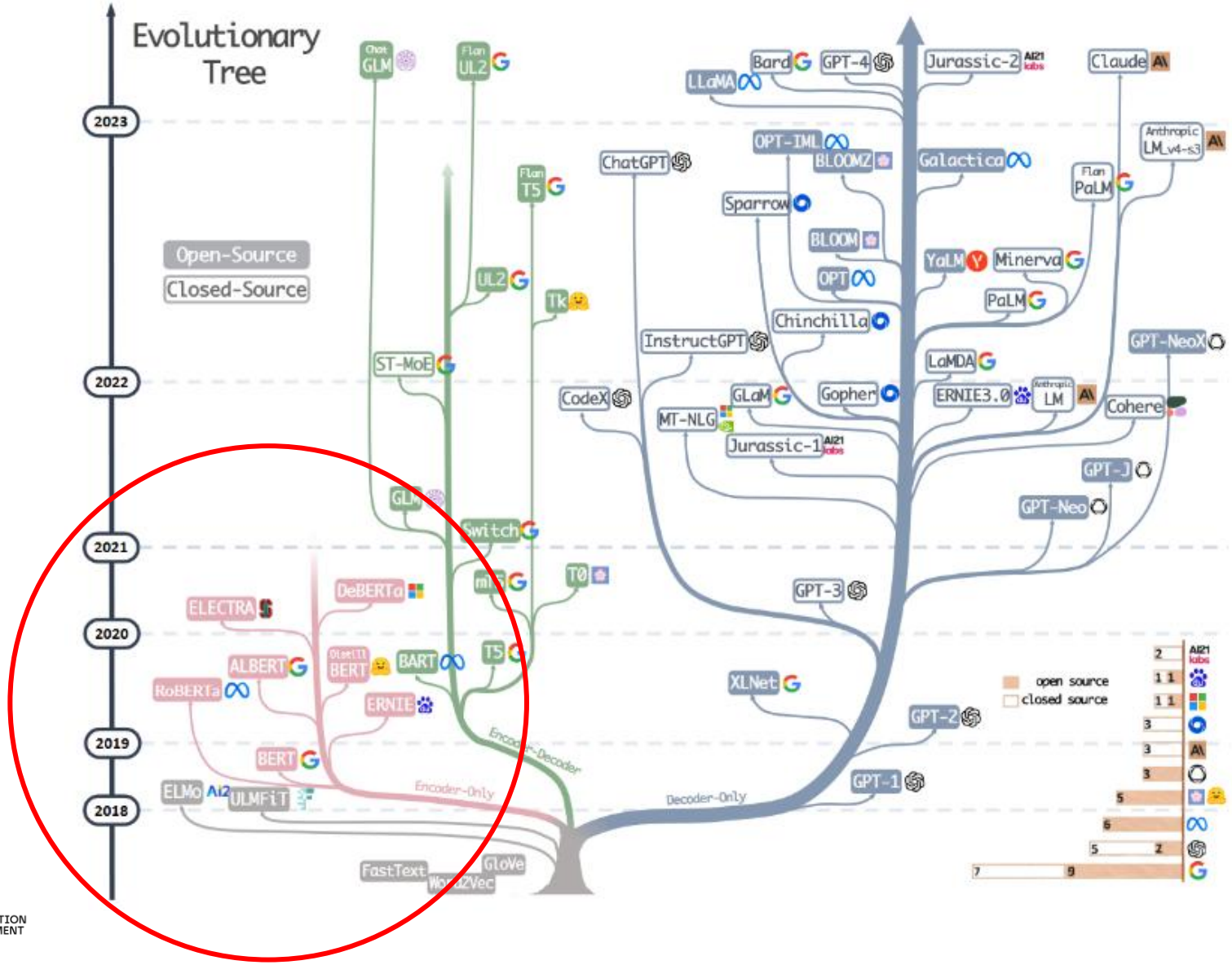
Encoder-only models read the full sentence at once, **capturing context from both previous and following words**.

They produce **rich, context-aware word and sentence representations**, usually stronger than static embeddings like Word2Vec or GloVe.

These **representations can be reused in downstream tasks** such as classification, sentiment analysis, NER, similarity, clustering, and retrieval (**Transfer Learning**).

Generating Word Representations with Transformers

Transformer Encoders



Generating Word Representations with Transformers

Transformer Encoders

Hugging Face list of Transformers here

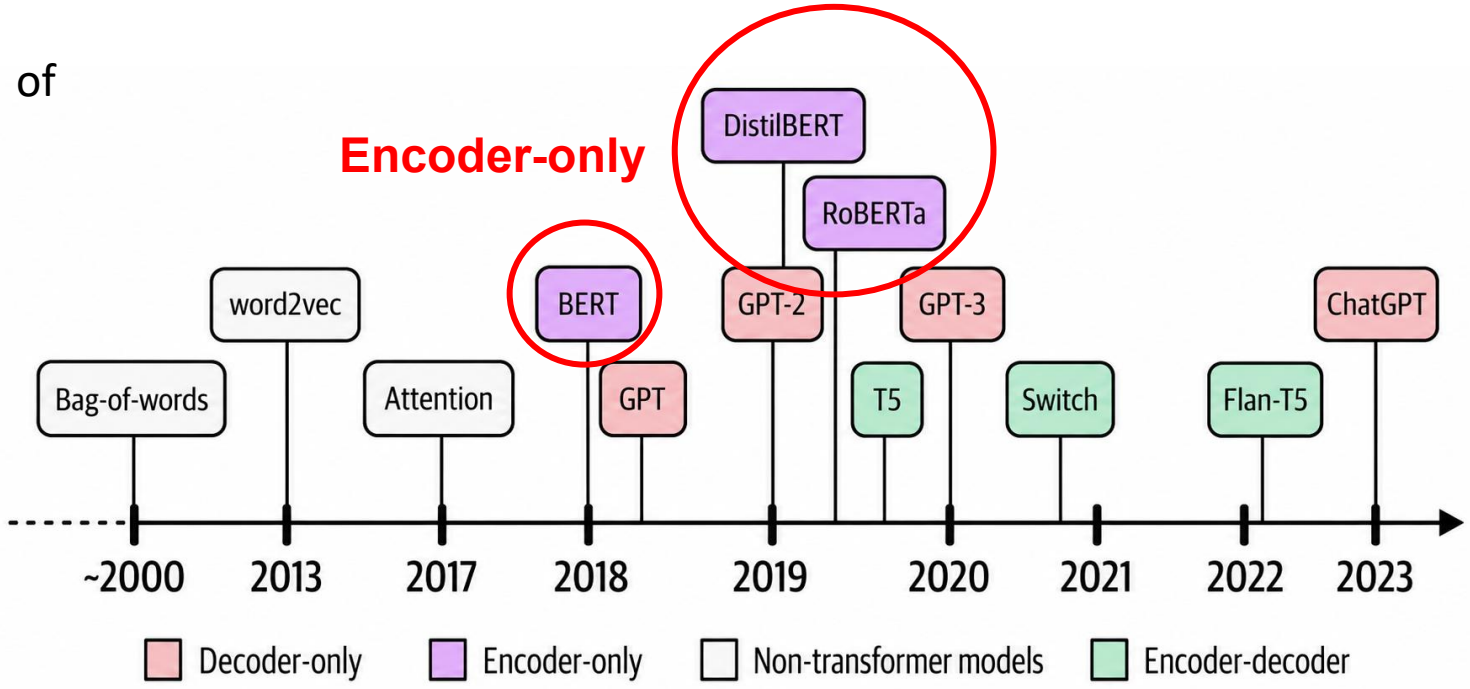


Figure 1-1. A peek into the history of Language AI.

Alammar, J., & Grootendorst, M. (2024). **Hands-On large language models: Language Understanding and Generation**. Sebastopol : O'Reilly Media.

Generating Word Representations with Transformers

Transformer Encoders

BERT (Bidirecional Encoder Representations from Transformers), developed by Google, is probably the most famous **Encoder-only Transformer** model.

BERT ($BERT_{Large}$) model uses **24 encoder layers**, referred to in the paper as Transformer blocks.

BERT uses **16 self-attention heads**, allowing the model to capture multiple contextual relationships in parallel.

It has a hidden size of **1024 units**, making its internal representations larger and more expressive. This means that **each token is represented by a 1024-dimensional contextual embedding** produced by the final encoder layer.

The representations generated by **BERT** can then be **fed to another model as word embeddings**

BERT

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). **BERT: Pre-training of deep bidirectional transformers for language understanding.** Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), 4171–4186. https://doi.org/10.18653/v1/N19-1423

Transfer learning is the process of **reusing knowledge learned from one task to improve performance on another task**.

In NLP, models such as BERT are first **pre-trained on massive text corpora to learn general language patterns**. They are called **Pre-trained Language Models** (PLMs)

This models can be called **Foundation models** - very **large pre-trained models trained on broad and diverse datasets at massive scale**.

The **pre-trained model is then adapted (fine-tuned) to a specific downstream task** using a smaller labeled dataset.

BERT was **pre-trained on large unlabeled datasets**, including **English Wikipedia** and the **BookCorpus** collection of books.

The model **learns language patterns using self-supervised tasks** such as **Masked Language Modeling (MLM)** and **Next Sentence Prediction (NSP)**.

- In **MLM**, during training, **random words are masked, and BERT learns to predict them** using context from both directions of the sentence.
- In **NSP**, BERT receives two sentences as input and **predicts whether the second sentence logically follows the first**.

The original BERT Large model contains approximately **340 million parameters**, with 24 encoder layers, 16 attention heads, and 1024 hidden dimensions.

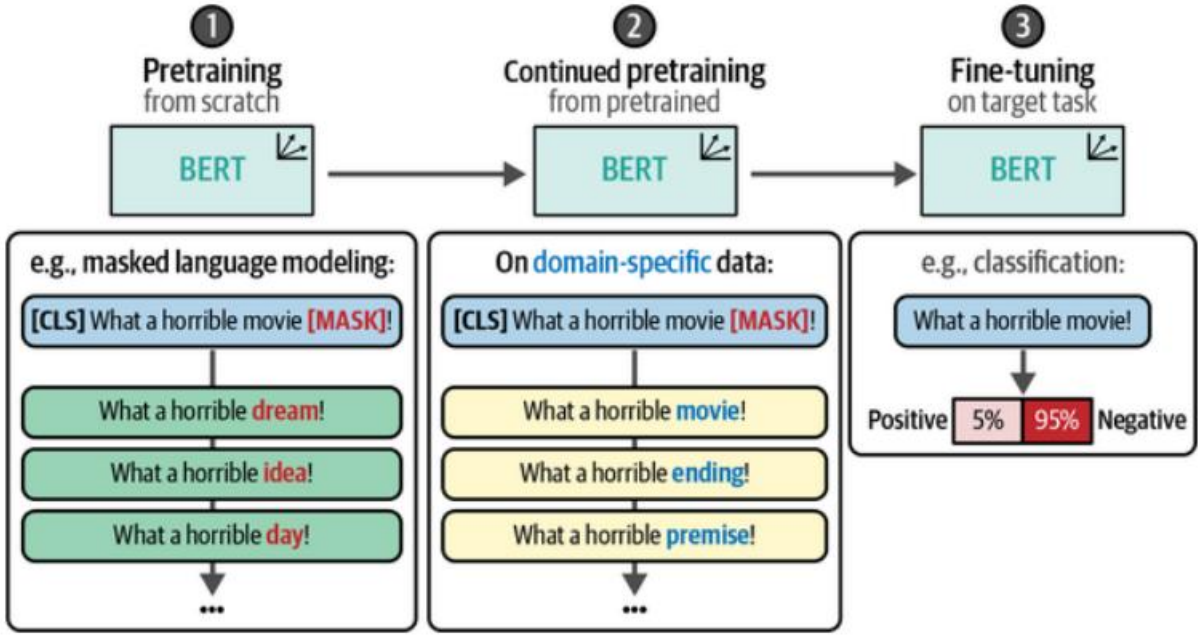
Stages of Training BERT

BERT **training is a two-step process**:

1. first **pretraining a model** (which is already done when we use it)

2. and then **fine-tuning it** for a particular task (there are a lot of models already fine tuned in hugging face for ex.)

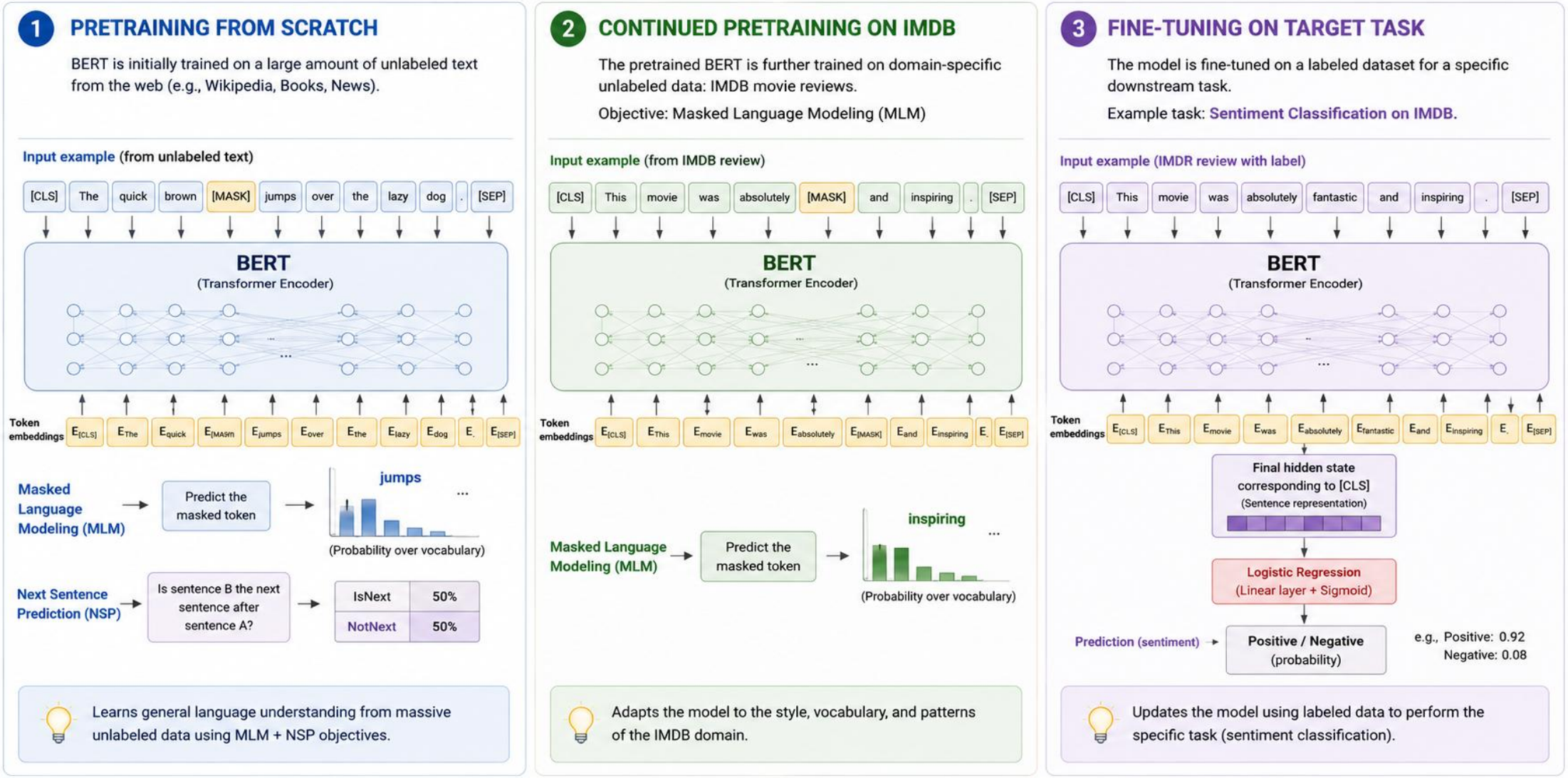
3. Finally, we can fine-tune it for a particular task, like sentiment analysis or predicting movie reviews



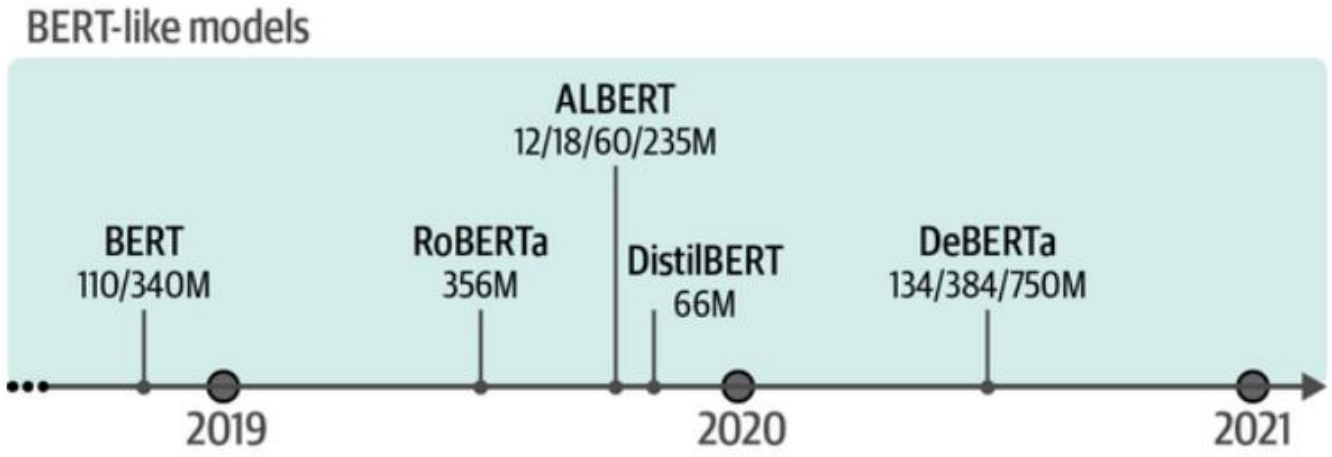
Alammar, J., & Grootendorst, M. (2024). **Hands-On large language models: Language Understanding and Generation**. Sebastopol : O'Reilly Media.

Training BERT

Transformer Encoders

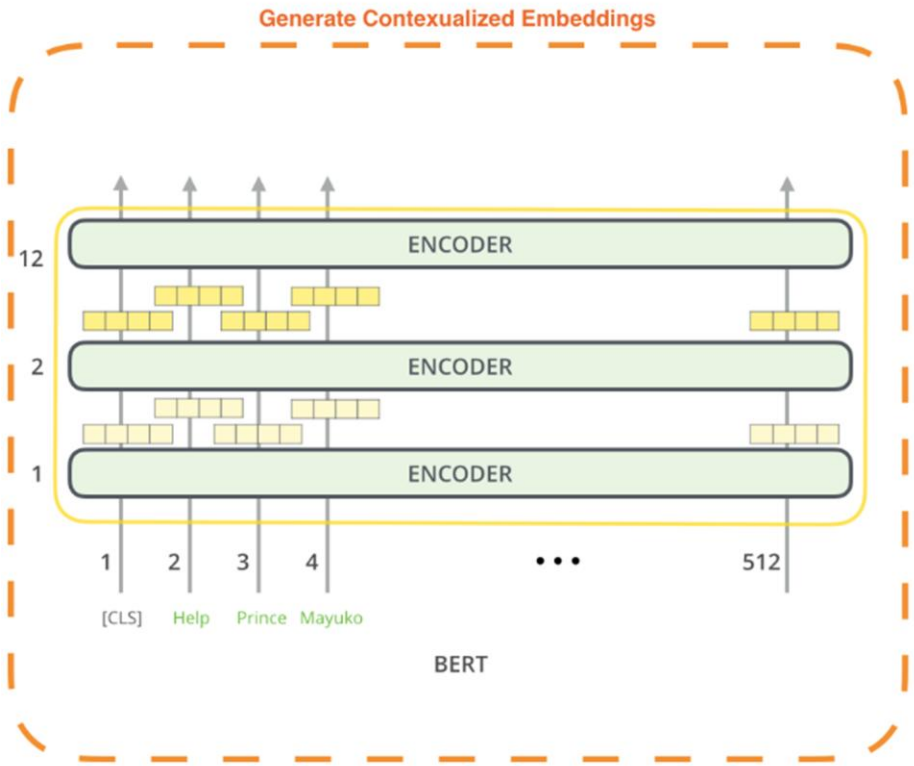


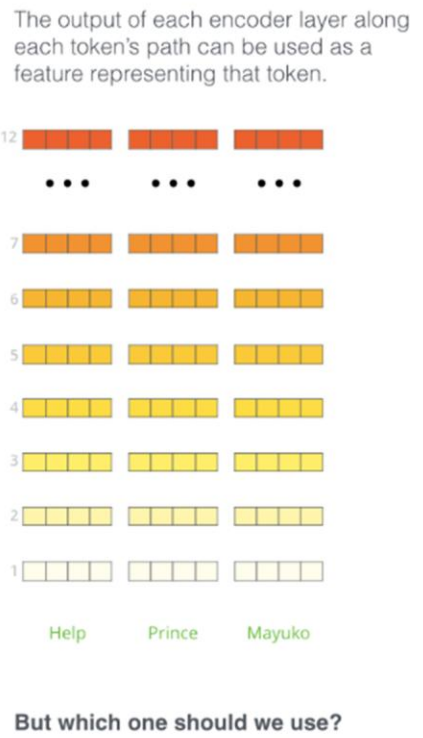
Over the years, **many variations of BERT** have been developed, including RoBERTa, DistilBERT, ALBERT, and DeBERTa, each trained in various contexts.



Alammar, J., & Grootendorst, M. (2024). **Hands-On large language models: Language Understanding and Generation**. Sebastopol : O'Reilly Media.

Since BERT produces multiple representations for the same word across layers, we must choose how to extract the final embedding.





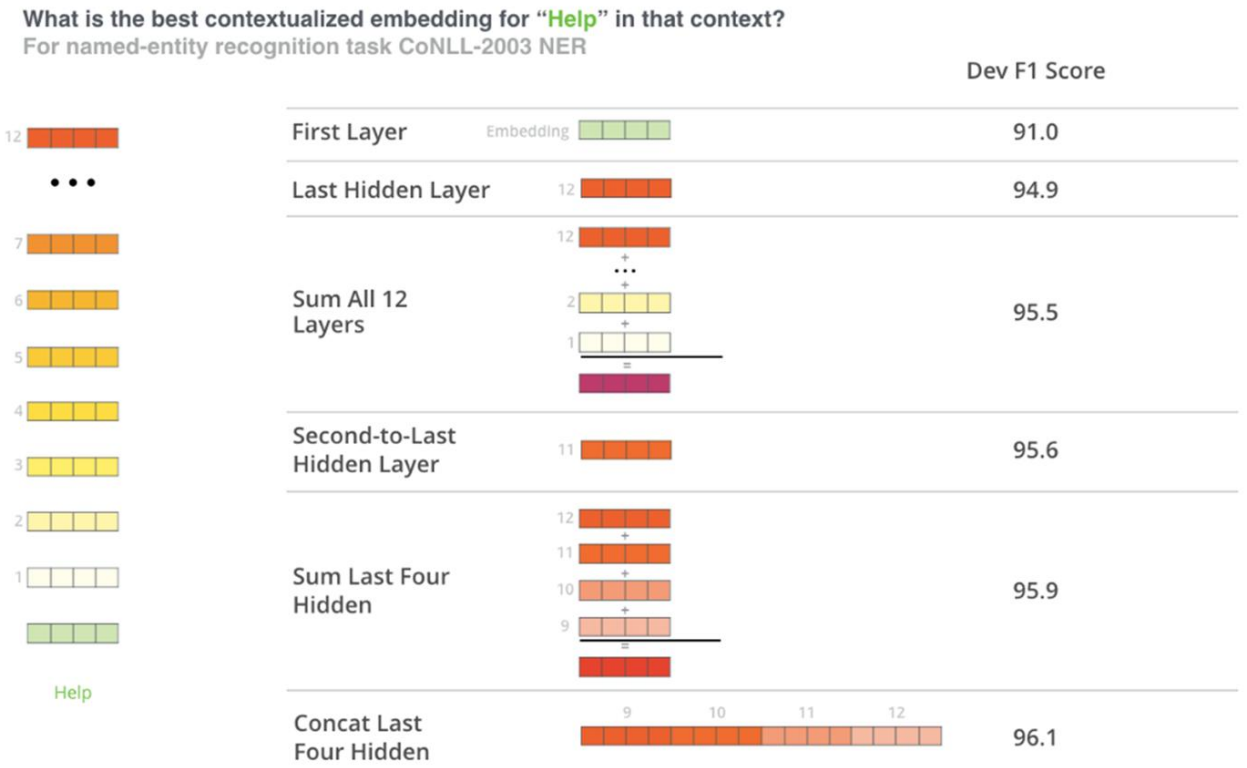
Word Embeddings

Transformer Encoders

To extract embeddings for each word, instead of a sentence level one, **we can aggregate information from all token embeddings using pooling** strategies.

Common approaches include:

- **Last Layer: use the final hidden representation of each token.**
- **Sum all layers**:sum the token representations from all Transformer layers.
- **Sum last four**: sum the token representations from the final four layers.
- **Concat last four**: join the final four token representations into one larger embedding vector.



*Learn more about **Pooling** here

Sentence Embeddings

Transformer Encoders

BERT adds a special token called **[CLS]** at **the beginning of every input sequence***.

During self-attention, [CLS] can attend to all words in the sentence, while all words can also attend to [CLS]. Across multiple Transformer layers, **the [CLS] embedding gradually becomes a contextual summary of the entire sequence**.

During pre-training, objectives such as **Next Sentence Prediction (NSP)** , but also during **fine tuning**, we can explicitly force **[CLS] to capture sentence-level information**.

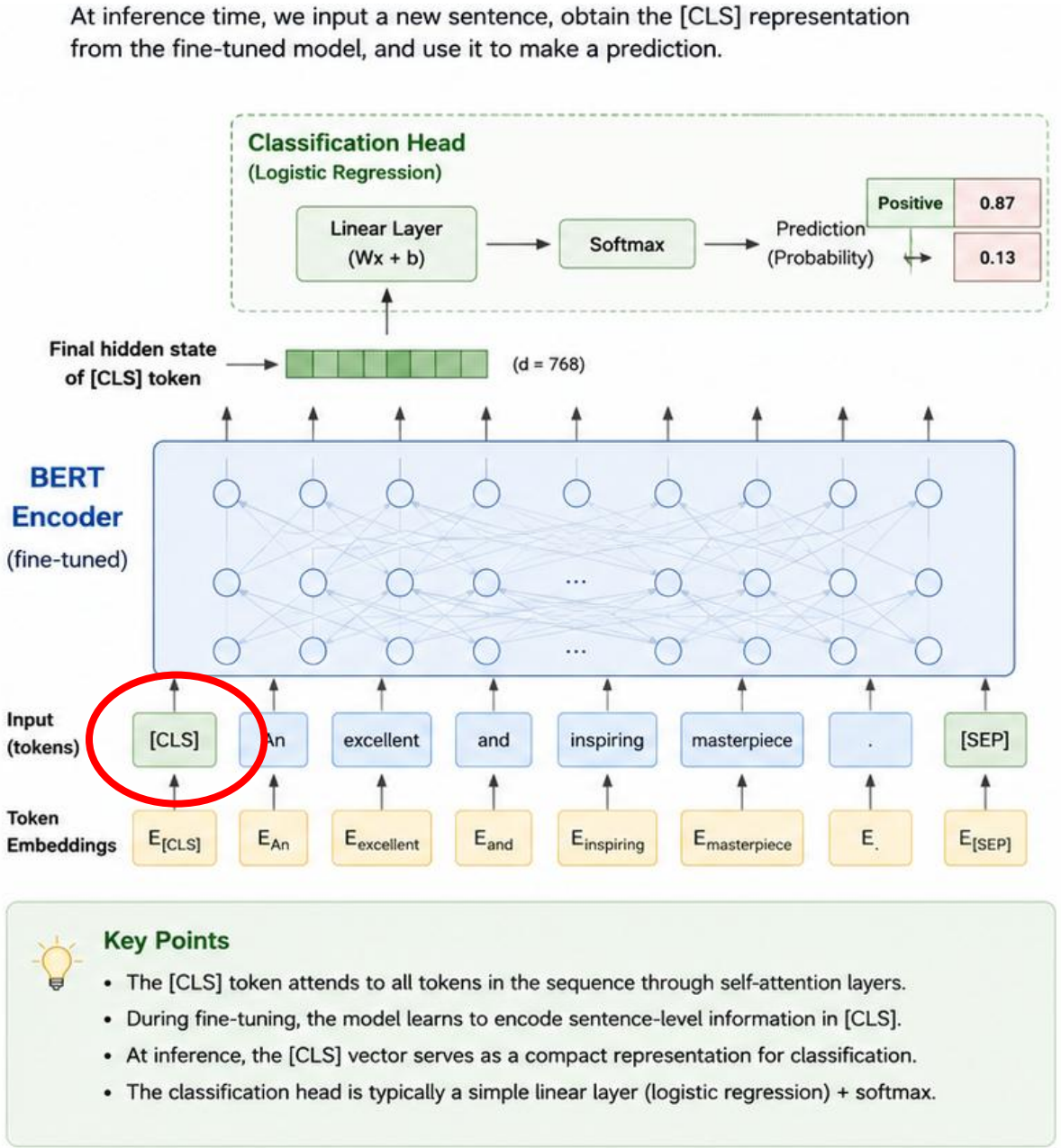
After the final encoder layer, **BERT outputs one contextual embedding per token**. For sequence classification tasks, the **final embedding corresponding to [CLS] is usually selected as the sentence representation**.

This vector is then **fed into a simple classifier**, typically a linear layer followed by Softmax or a Logistic Regression.

*Learn more about **[CLS]** token here

Classification with BERT

Transformer Encoders



*Learn more about **[CLS]** token here

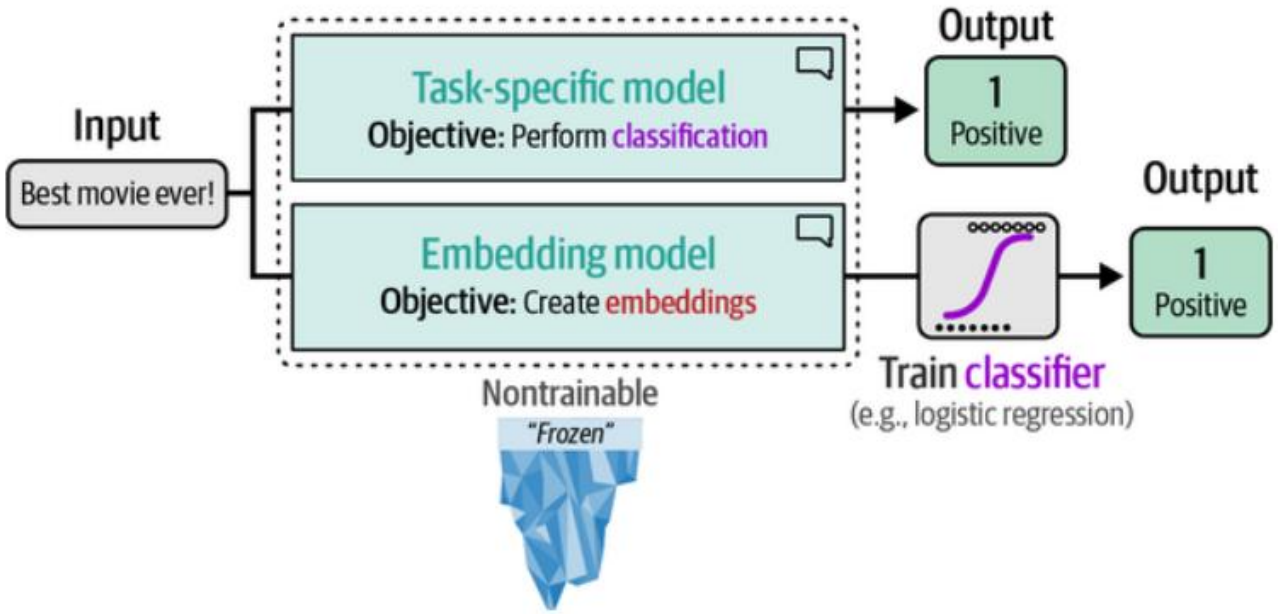
Classification with BERT

Transformer Encoders

We can **perform classification directly with a task-specific model** or indirectly with general-purpose embeddings

A **task-specific model** is a representation model, such as BERT, trained for a specific task, like sentiment analysis

BERT can also be used to **generate general-purpose context embeddings** that can be used for a variety of tasks not limited to classification, like semantic search



Alammar, J., & Grootendorst, M. (2024). **Hands-On large language models: Language Understanding and Generation**. Sebastopol : O'Reilly Media.

Bibliographic references

Dan Jurafsky and James H. Martin. **Speech and Language Processing** 3rd ed. **Chapter 12** https://web.stanford.edu/~jurafsky/slp3/

Alammar, J., & Grootendorst, M. (2024). **Hands-On large language models: Language Understanding and Generation**. Sebastopol : O'Reilly Media

Alammar, J. (2018, May 9). **Visualizing a neural machine translation model (Mechanics of seq2seq models with attention)**. Jay Alammar. https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/

Alammar, J. (2018, June 27). **The illustrated transformer**. Jay Alammar. https://jalammar.github.io/illustrated-transformer/

Alammar, J. (2018). **The illustrated BERT, ELMo, and co. (How NLP cracked transfer learning)**. Jay Alammar – Visualizing machine learning one concept at a time. https://jalammar.github.io/illustrated-bert/

NOVA IMS INFORMATION MANAGEMENT SCHOOL

Thank you!

Acreditações e Certificações da NOVA IMS

eduniversal · QS STARS · ABET Computing Accreditation Commission · iSchools · EMOS · UNIGIS · USGIF · Erasmus Mundus Programme Education and Culture DG · apcer · IQNet

Cofinanciado por

PORTUGAL 2030 · 2030 Lisboa Programa Regional de Lisboa · Co-funded by the European Union · fct Fundação para a Ciência e a Tecnologia · PRR Plano de Recuperação e Resiliência · República Portuguesa · Financiado pela União Europeia NextGenerationEU